Supplementary Material for "A Survey of Small Language Models for Agentic AI: Architectures, Compression, Deployment, and Evaluation"

MOHAMMED GAMAL RAGAB*, Universiti Teknologi PETRONAS, Malaysia
HITHAM ALHUSSIAN*, Universiti Teknologi PETRONAS, Malaysia
SAID JADID ABDULKADIR, Universiti Teknologi PETRONAS, Malaysia
W. ELTAYEB AHMED, Imam Mohammad Ibn Saud Islamic University (IMSIU), Saudi Arabia

This electronic supplement contains the material referred to from the main text as Supplementary Sections S1–S18: the reporting dimensions and artifact vocabulary; the inclusion criteria, source verification, released artifacts, and update policy; the key model families; the technical background; the inventory figures; the taxonomy facets and architectural detail; compression methods and distillation variants; the audit protocol and coverage ledger; reference tables; deployment platforms; the conversion pipeline and routing designs; domain applications; model-level challenges; open research directions; the environmental accounting boundary; data provenance; and the generated variant inventory.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; • **Computer systems organization** → *Embedded and cyber-physical systems.*

Additional Key Words and Phrases: Small language models, agentic AI, model inventory, evaluation reporting, model compression

S1 Reporting Dimensions and Artifact Vocabulary

S1.1 Reporting Dimensions

A single parameter count is not enough to determine whether a model will run on a given device. Table S1 lists the dimensions this survey reports where the primary source supplies them, and states which of them each of our claims depends on. Where a source omits a dimension, the entry is marked as not reported rather than estimated.

S1.2 Model-Artifact Vocabulary

Because the inventory counts artifacts rather than papers, we also fix the following. A *family* is a named line of models sharing an architecture and a training recipe (Qwen2.5, Gemma 3). A *variant* is one released parameter size within a family. Where a vendor publishes the base and the post-trained checkpoint of that size in one release, the inventory records one variant and marks its post-training status as "both" (63 of the 209); where only one is published, or the two were released separately, the status is recorded as that one. The post-training status is therefore a field of the variant, not a second count. A *checkpoint* is a specific set of weights published under a version

*Corresponding authors.

Authors' Contact Information: Mohammed Gamal Ragab, mohammed.ragab@utp.edu.my, Department of Computer and Information Sciences, Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar, Perak, Malaysia; Hitham Alhussian, seddig.alhussian@utp.edu.my, Department of Computer and Information Sciences, Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar, Perak, Malaysia; Said Jadid Abdulkadir, saidjadid.a@utp.edu.my, Department of Computer and Information Sciences, Center for Research in Data Science (CeRDaS), Universiti Teknologi PETRONAS, Seri Iskandar, Perak, Malaysia; W. Eltayeb Ahmed, waahmed@imamu.edu.sa, Department of Mathematics and Statistics, College of Science, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh, Saudi Arabia.

2026. ACM 1557-7341/2026/10-ART

Table S1. Reporting dimensions for a small language model, the deployment question each answers, and the coverage achieved across the 209-variant inventory. "Derived" means the value follows arithmetically from others rather than being reported.

Dimension	Question it answers	Bounds	Inventory coverage
Total parameters	How large is the download?	C2	209/209
Active parameters per token	What does one token cost?	C1	36/209 stated below total; else = total
Weight memory at a precision	Will the weights fit?	C2	Derived
Peak runtime memory	Will it fit while running?	C2	Not reported by vendors
FLOPs per token	Compute cost per token	C1	Derived (approximate)
KV-cache size per token	Cost of long context	C1, C2	Derived from head config
Context length	Largest usable input	–	135/209
Target hardware	What did the vendor test on?	–	Vendor-stated only

identifier; a variant may be revised through several checkpoints, and the inventory records the variant, not each checkpoint. A *release* is a dated publication event covering one or more variants. A *pretrained* (or base) model is the artifact produced by pretraining alone; an *instruction-tuned* model has undergone supervised instruction fine-tuning and usually a preference-optimization stage; a *reasoning-tuned* model has additionally been trained, by distillation of long chain-of-thought traces, reinforcement learning, or both, to emit extended intermediate reasoning before its answer. These three post-training statuses are not interchangeable for evaluation purposes, which is why an evaluated artifact must be identified to the checkpoint even though the inventory counts variants, and conflating them is one of the specific defects in the public record documented in Section 5.2.

Two conventions follow from this vocabulary and are used consistently throughout. Family names are written as their developers write them and with a space before the version, as in Gemma 3, Qwen2.5, Phi-4, and Llama 3.2; the original 2023 LLaMA release keeps its own capitalization, since Meta renamed the line to "Llama" from Llama 2 onward. Checkpoint identifiers are written in the hyphenated form used on the model hub, as in `Gemma-3-4B-it` or `Qwen2.5-7B-Instruct`, and are used wherever the specific artifact rather than the family is meant.

S2 Inclusion Criteria, Source Verification, Released Artifacts, and Update Policy

S2.1 Inclusion Criteria and Appraisal

Inclusion criteria for the technique literature were: (T1) the work concerns models at or below roughly twenty billion parameters, or a compression, adaptation, routing, or deployment technique whose evaluation includes that regime; (T2) it is a peer-reviewed paper, an archived preprint, or a vendor technical report with a stable identifier; and (T3) it reports enough methodological detail to determine what was measured.

Inclusion criteria for model variants were: (M1) a source from the developer (technical report, paper, model card, model repository, or release announcement) that states the parameter count and architecture; (M2) a public artifact or a documented evaluation card; (M3) satisfaction of C1, with C2 status recorded separately; and (M4) a dated release. Fine-tunes and merges of another party's checkpoint were excluded unless their developer released a technical report describing its own training recipe (as for the DeepSeek-R1-Distill models), as were quantized re-uploads, API aliases of a listed checkpoint, and variants differing only in serialization format.

What is computed and what is decided. C1, C2, M1, and M4 are computed from recorded fields by `apply_inclusion_rules.py`: M1 from the stated parameter count and the field `developer_source_foun` written by the source check below, and M4 from `release_date_precision`. Three kinds of exclusion are named editorial decisions listed in the script, because they cannot be computed from

fields: encoder-decoder architecture (Flan-T5), derivative fine-tunes without a technical report (Nous Hermes 2 Mixtral, Vicuna-7B v1.5, Vicuna-13B, Jan-v1-4B), and an API alias (Mistral Tiny). Records whose size is undisclosed cannot be classified against C1 or C2; all of them, whatever their marketing (for example GPT-4o mini, Amazon Nova Micro and Lite, Claude Haiku, Gemini Flash-Lite, Nex-N2-Mini), are kept in the comparison part of the file with class `undisclosed` and are neither counted nor plotted as small language models (SLMs) or large language models (LLMs). Release dates are recorded at the precision their sources give: day for 174 of the 209 included records, month for 22, and year for 13.

Appraisal of empirical works. Works relied on for quantitative claims were appraised on whether they name the harness, state the evaluation conditions, report a matched baseline, and identify the evaluated checkpoint; the outcome, together with the reporting party's independence, is the evidence grade of Section 5.1.2.

S2.2 Source Verification

On 8 October 2026, `verify_sources.py` checked every included and excluded record, and every comparison record of stated size, against the Hugging Face Hub API: for each Hub repository a record links, it recorded whether the repository exists, its owner, creation date, licence tag, and the parameter count of its safetensors weights, and it classified the record's reference as a developer page, a paper, a Hub model page, or a secondary source (a model aggregator or press article). The result is written into each record (`source_check`, `developer_source_found`) and into `source_check_log.csv`; Figure S1 places the check in the build process. Of the 209 included records, 197 link an existing repository owned by the developer and 12 link the developer's own announcement, paper, or code repository.

The check led to the following corrections, applied by `fix_records_v3_6.py` and listed in the file's metadata. Twelve records whose reference was a model aggregator or a press article (the DeepSeek V4 Flash, Ling 3.0 Flash, Laguna, North Mini Code, Inkling Small, Nex-N2-Pro, Dots3-Note Preview, and Granite 4.0 Nano records) were re-sourced to the developer's Hub repository, with the original reference kept as `discovered_via`; Ling 3.0 Flash Fin, recorded as API-only, is open-weight under MIT. Ling 3.0 Flash Sante, for which no developer source exists, fails M1 and was excluded. Voxtral Mini, previously excluded as undisclosed with a link to a different model, is the open-weight Voxtral Mini 3B (4.7B including its audio encoder, Apache 2.0) and was included. The BitNet b1.58 record, which described the 2024 paper but linked the 2025 checkpoint, now describes the released 2B4T model. The SmolLM2 records, which linked the SmolLM3 page, the Mistral 7B v0.3 release date (2024, not 2023), and the Granite 4.1 references were corrected. For nine records the ratio of the Hub parameter count to the vendor's nominal size falls outside 0.8–1.25 (for example Gemma 2-2B, nominal 2B, Hub 2.6B; Qwen2.5-Omni-7B, Hub 10.7B including its audio components; BitNet b1.58 2B4T, whose ternary weights are stored packed); the recorded size is the vendor's statement, the Hub count is kept in the log, and none of the differences changes a record's C1 or C2 status.

S2.3 Count History

Earlier drafts reported 126 variants. Version 3.3 of the inventory added 27 releases from May to September 2026 (153). Version 3.4 placed records by the rule rather than by how they had been filed: 64 models meeting C1 had been filed outside the SLM inventory and two 24B dense models inside it (215). Version 3.5 corrected a parser that had read the total in place of the active count for five legacy mixture of experts (MoE) records and removed eleven records by the inclusion criteria (209: 178 SLMs and 31 small-active-parameter MoEs). Version 3.6 applied the source verification above: Voxtral Mini 3B entered as an SLM and Ling 3.0 Flash Sante left, and the four records of undisclosed

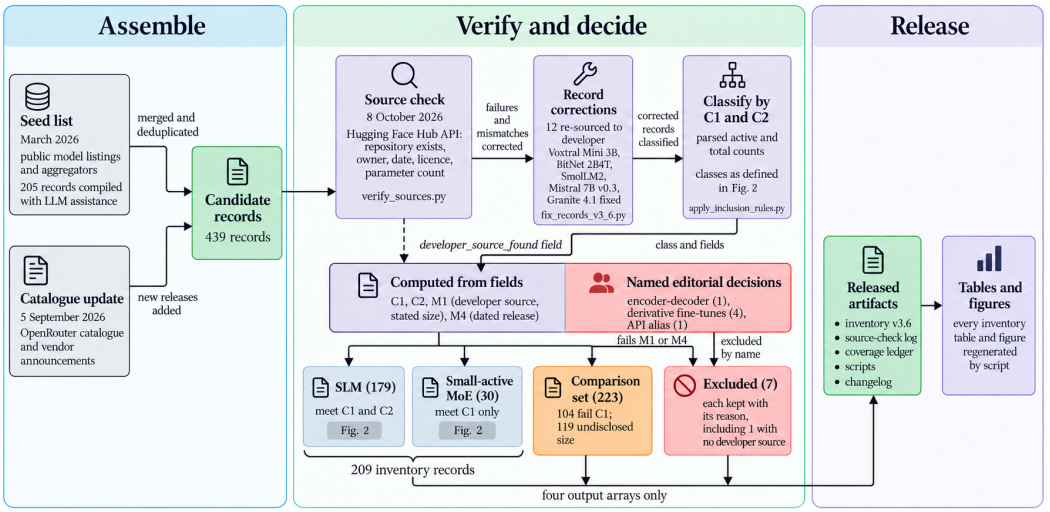


Fig. S1. How the inventory was assembled, verified, and filtered. Secondary listings only seeded the candidate list; every record was then checked against its developer source and corrected where needed. C1, C2, M1, and M4 are computed from recorded fields; the encoder-decoder, derivative, and alias exclusions are named editorial decisions. All four arrays, the source-check log, the coverage ledger, the scripts, and the changelog are released.

size that version 3.5 had excluded as "vendor-positioned small" were moved to the comparison set with every other undisclosed record (209: 179 SLMs and 30 small-active-parameter MoEs; 223 comparison records, of which 104 are of disclosed size and fail C1 and 119 are undisclosed; 7 excluded). The changelog in the released file names every record added, moved, or removed at each step.

S2.4 Released Artifacts

The following are released with the paper: the inventory `merged_llm_database.json` (version 3.6), with one record per variant carrying its parameter counts, C1/C2 status, class, inclusion decision and reason, developer source, source-check result and date, and the excluded and comparison records alongside; the source-check log `source_check_log.csv`; the audit coverage ledger `audit_coverage_ledger.csv` (Supplementary Section S9.2); the scripts that correct and verify records, apply the inclusion rule, build the ledger, and generate every inventory table and figure; and the changelog. They are provided as supplementary files with the submission and will be deposited in a versioned archival repository under a persistent identifier, so that the exact snapshot underlying this paper, and any later one, can be retrieved.

S2.5 Snapshot and Update Policy

This paper reports model releases through 4 September 2026, as recorded in version 3.6 of the inventory (8 October 2026). Releases appearing between submission and acceptance are added to the released inventory and recorded in the changelog, and are discussed in the text only where they change a conclusion; tables and figures are regenerated from the inventory at the last revision before acceptance. Works appearing during review that bear on RQ2–RQ5 are added to the synthesis, as was done for one study published after the snapshot [71]. The released artifact is the authority on what the inventory contained at any given date.

S3 Key SLM Families

S3.1 Microsoft Phi Series

The Phi series represents Microsoft's systematic exploration of data-quality-driven model development. Phi-1 (1.3B) [53] demonstrated strong code generation performance through training on synthetic "textbook-quality" data. Phi-2 (2.7B) [97] extended this to general language understanding. Phi-3-mini (3.8B) [2] formalized the approach with 3.3 trillion training tokens, reporting 68.8% on Massive Multitask Language Understanding (MMLU) (5-shot) and 82.5% on Grade School Math 8K (GSM8K) (8-shot, chain-of-thought) for the instruction-tuned checkpoint (C). Phi-3.5-mini [94] added 128K context length support with improved multilingual capabilities. Phi-4 (14B) [3] introduced a fundamentally enhanced training pipeline incorporating ∼40% synthetic data, reporting 84.8% on MMLU and 80.4% on MATH under OpenAI's simple-evals settings, and reporting that it exceeds its teacher GPT-4o on several STEM benchmarks (C). Phi-4-mini (3.8B) [96] updated the architecture with grouped-query attention (GQA) (24 query, 8 key–value (KV) heads), a 200K-token vocabulary, and training on 5 trillion tokens, reporting 88.6% on GSM8K (8-shot, chain-of-thought; C). Phi-4-reasoning [1] is Phi-4 after supervised fine-tuning on curated prompts with o3-mini reasoning demonstrations; Phi-4-reasoning-plus adds a short phase of outcome-based reinforcement learning. Both are reported to outperform DeepSeek-R1-Distill-Llama-70B and to approach the full DeepSeek-R1 on AIME 2025 (C).

S3.2 Google Gemma Series

Google's Gemma family [44] derives from the Gemini model architecture and training infrastructure. Gemma 1 offered 2B and 7B variants trained on 3T and 6T tokens respectively, using Multi-Query Attention (MQA) for the 2B model and standard Multi-Head Attention for the 7B. Gemma 2 [43] introduced significant architectural improvements: local-global interleaved attention layers, logit soft-capping, and knowledge distillation for the 2B and 9B variants (from a larger teacher model). The pre-trained Gemma 2-9B reports 71.3% on MMLU (5-shot; C). Gemma 3 [45] further advanced the architecture with a 5:1 local-to-global attention interleaving ratio (reducing the sliding window from 4096 to 1024 tokens) and replaced the logit soft-capping of Gemma 2 with QK-norm. Gemma 3 distills all model sizes from larger teachers and introduces a 1B variant whose instruction-tuned checkpoint reports 48.0% on MATH, close to the 49.4% of the instruction-tuned Gemma 2-9B with 9× fewer parameters (C).

S3.3 Alibaba Qwen Series

The Qwen2 [138] and Qwen2.5 [112] families from Alibaba are trained on up to 18 trillion tokens, far beyond Chinchilla-optimal ratios. Qwen2.5 offers models at 0.5B, 1.5B, 3B, 7B, 14B, 32B, and 72B parameters, all sharing a consistent GQA-based architecture with rotary position embeddings (RoPE) and SwiGLU activations. The pre-trained 3B model reports 65.6% on MMLU (5-shot), 79.1% on GSM8K, and 42.6% on MATH (both 4-shot) (C); its developers credit the STEM and code share of the training data for its mathematics and coding results.

S3.4 Other Significant SLM Families

TinyLlama (1.1B) [143] adopted the Llama 2 architecture and tokenizer and trained on about 1 trillion tokens of SlimPajama and StarCoder data for roughly three epochs, showing that extended training of very small models continues to pay. Mistral 7B [62] combined grouped-query attention, introduced by Ainslie et al. [5], with sliding-window attention in a 7.3B-parameter model, and reports outperforming Llama 2 13B while being 1.8× smaller. Meta's Llama 3.2 [91] released 1B and 3B models for edge and mobile devices, derived from Llama 3.1 8B by pruning and distillation,

trained on up to 9 trillion tokens with 128K context. SmolLM2 (135M, 360M, 1.7B) [6] from Hugging Face trained on 11 trillion tokens of carefully curated data. H2O-Danube (1.8B) [120] introduced sliding window attention in a compact model trained on 1 trillion tokens. OLMo 2 (1B, 7B, 13B) [127] from AI2 was released as a fully open model (data, code, weights, and training logs), achieving competitive performance through QK-norm and carefully staged training. StableLM-2 (1.6B) [124] was trained on about 2 trillion tokens (counting repeated epochs) of multilingual data. The BLOOM family (560M, 1.1B, 1.7B, 3B, 7.1B) [19] remains notable as a multilingual model trained on 46 languages using ALiBi positional encoding, though its 350B-token training budget is modest by current standards.

S4 Training, Compression, and Deployment Background

S4.1 Knowledge Distillation and Model Compression

Knowledge distillation, first formalized by Hinton et al. [59], transfers knowledge from a large "teacher" model to a smaller "student" by training the student to match the teacher's soft probability distributions (logits) rather than hard labels. This approach preserves the teacher's learned representations of inter-class similarities, information that is lost in one-hot labels, and has proven remarkably effective in the LLM era. Modern distillation strategies for language models encompass several complementary approaches.

Logit-based distillation trains the student to minimize the KL divergence between its output distribution and the teacher's softened distribution, parameterized by a temperature τ. Google's Gemma 2 [43] and Gemma 3 [45] employ this approach extensively: all Gemma 3 model sizes, including the 27B variant, are trained using knowledge distillation from a larger teacher. The distillation loss is combined with a standard next-token prediction loss, with the relative weighting serving as a key hyperparameter. Feature-based distillation [116] additionally aligns intermediate representations between teacher and student, capturing structural knowledge that may not be fully expressed in output logits. Microsoft's Orca 2 [99] demonstrated a variant where a smaller model is taught to use different reasoning strategies depending on task complexity, outperforming models with 5–10× more parameters on reasoning benchmarks.

Synthetic data generation represents an increasingly dominant form of implicit distillation. Rather than direct logit matching, a powerful LLM (e.g., GPT-4, Claude 3.5) generates high-quality training examples that are then used to train smaller models. The Phi series pioneered this approach with the "Textbooks Are All You Need" methodology [53], generating synthetic textbook-style content with carefully controlled complexity and diversity. Phi-4's training data is reported to be about 40% synthetic, generated across 50 dataset types including multi-agent conversations and structured reasoning chains [3] (C); its developers attribute much of its STEM performance to this data, an attribution that has not been tested in a controlled comparison against web-corpus training at equal compute. Post-training quantization, pruning, and adaptation methods are reviewed in Supplementary Section S7; together with distillation during training they form the usual SLM optimization pipeline.

S4.2 Efficient Fine-Tuning Methods

Full fine-tuning updates all model parameters using task-specific data, which for a 7B-parameter model can require tens of gigabytes of accelerator memory once gradients and optimizer states are included. This cost has made parameter-efficient fine-tuning (PEFT) central to the practical adaptation of SLMs, particularly when the target deployment environment is a single GPU or an edge-oriented workstation.

low-rank adaptation (LoRA) [60] addresses this constraint by freezing the pretrained backbone and learning low-rank update matrices for selected linear layers. For a weight matrix $W_0 \in \mathbb{R}^{d\times k}$, the update is parameterized as $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d, k)$. This drastically reduces the number of trainable parameters while preserving the ability to specialize a model for downstream tasks. quantized low-rank adaptation (QLoRA) [32] extends this idea by quantizing the frozen base model to 4-bit NormalFloat (NF4) precision and training the adapters in higher precision, allowing models that would ordinarily require multi-GPU setups to be fine-tuned on a single high-memory accelerator.

Beyond LoRA-style adapters, the PEFT landscape includes prompt-based and prefix-based adaptation strategies that shift task information into learned soft prompts or attention prefixes rather than updating the backbone itself. These approaches can be attractive when multiple lightweight task specializations must be maintained simultaneously. However, they tend to underperform LoRA-family methods on complex reasoning tasks, where direct adaptation of attention and feed-forward projections provides a stronger inductive bias for task transfer.

The choice between prompting and fine-tuning therefore depends on task specificity, data availability, and deployment constraints. In-context learning offers zero-training adaptation and is well suited to broad tasks when labeled data are scarce, but it consumes context length and raises inference latency. By contrast, PEFT methods incur an upfront training cost yet reduce per-query overhead and typically yield larger gains on narrow or reasoning-heavy tasks. Prompting and QLoRA results are rarely reported under matched conditions, so they cannot be compared across models from the published record; measuring them is one of the verification studies proposed in Section 9.3.

S4.3 Edge AI and On-Device Inference

The edge AI inference stack has matured rapidly, transforming SLM deployment from a research exercise to a production-ready capability. The llama.cpp framework [47] is the most widely used runtime for CPU and mixed CPU-GPU inference, and the one against which the others in Table S8 position themselves, supporting the GPT-generated unified format (GGUF) quantization format with dozens of precision variants (Q2_K through Q8_0). Its ecosystem includes llamafile (single-executable packaging), Ollama (user-friendly serving), and extensive platform support spanning x86, ARM, Apple Silicon, and NVIDIA GPUs. Benchmarking on single-board computers shows that llamafile achieves up to 4× higher throughput and 30–40% lower power consumption than Ollama for equivalent models [104].

Meta's ExecuTorch [89] is an on-device runtime with a small base runtime and backends for CoreML, Qualcomm QNN, ARM, MediaTek, and Vulkan; Meta used it for the on-device measurements of its Llama 3.2 1B and 3B models [90]. MLC-LLM [100] deploys through ML compilation (TVM) to iOS, Android, web, and embedded GPUs. ONNX Runtime [93] provides cross-platform deployment with optimization passes for diverse hardware. WebLLM [101] runs inference in the browser with WebGPU and reports retaining up to 80% of native performance (C). Google's LiteRT (formerly TensorFlow Lite) targets mobile CPUs, GPUs, and neural processing units (NPUs) [50]; its vendor-reported speedups on Qualcomm NPUs are measured against CPU execution on the vendor's chosen models and are not comparable with the other figures here.

The fundamental constraint in edge deployment is memory bandwidth rather than compute. Flagship phones and mainstream laptops offer on the order of 50–100 GB/s of memory bandwidth, unified-memory laptop designs considerably more, and single-board computers well under 30 GB/s, versus 2–3 TB/s for datacenter GPUs [135]. Single-stream autoregressive decoding reads the full weight set once per generated token, so in that regime it is bandwidth-bound and quantization from 16 to 4 bits converts a storage saving into a decode-time saving roughly in proportion to the

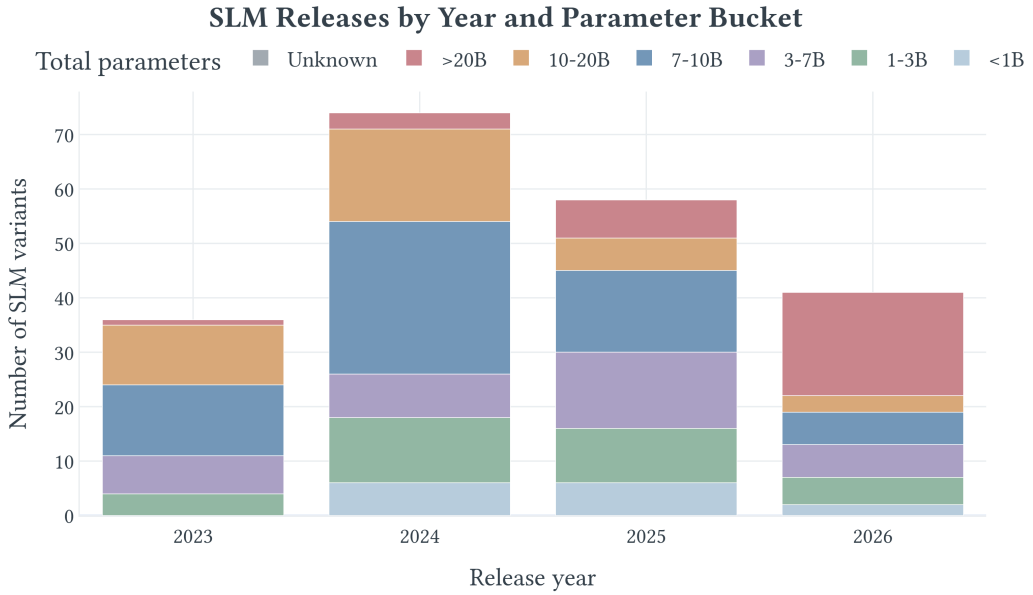


Fig. S2. Inventory variants by release year and total-parameter band (2026 to 4 September). The 7–10B band is the largest in 2023, 2024, and 2025; the sub-1B and 1–3B bands appear in numbers from 2024; in 2026 the largest band is the one above 20B total, which consists of small-active-parameter MoEs.

bandwidth freed. The proportionality is regime-dependent rather than general: it weakens under batching as arithmetic intensity rises, largely disappears during the compute-bound prefill phase, and is reduced further on hardware without native low-precision kernels. Section 6.2 sets out the conditions in full. Because single-stream interactive use is the dominant on-device regime, INT4 quantization is nevertheless a performance enabler there and not only a storage optimization.

S5 Inventory Figures

The figures below are generated from version 3.6 of the released inventory (releases through 4 September 2026) and are referred to from Section 3 of the main text. Unless stated otherwise, they cover the 209 inventory records, that is, the 179 SLMs and the 30 small-active-parameter MoEs. Figure S2 gives releases by size band, Figure S3 the size span of the principal families, Figure S4 context length by year, Figure S5 the dense and MoE split, and Figure S6 supported modalities.

S6 Taxonomy Facets in Detail

This section expands Section 3 of the main text.

S6.1 Architecture

Dense transformers. Most SLMs use a dense decoder in which every parameter is active for every token, from SmolLM2-135M [6] to 7–14B models such as Mistral 7B [62] and Phi-4. SmolLM3-3B [14] uses grouped-query attention with no positional encoding in a share of its layers and a three-stage curriculum over 11.2T tokens; Qwen3's dense line [137] spans 0.6B–32B with a switch between thinking and non-thinking modes. Three refinements recur. Grouped-query attention [5],

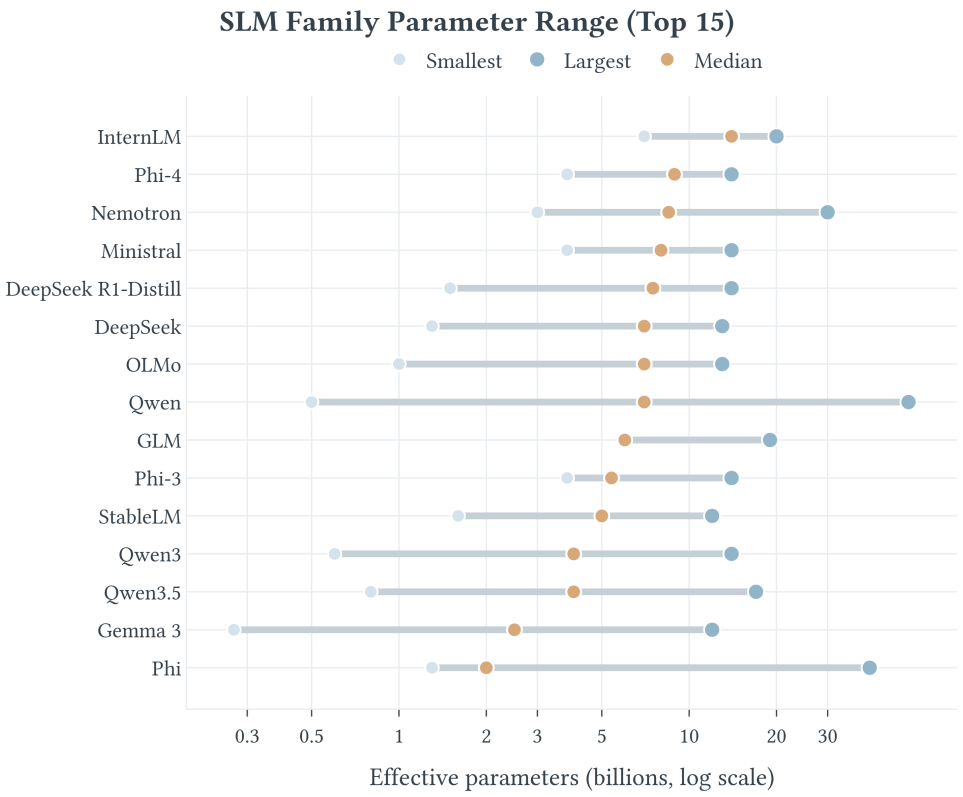


Fig. S3. Parameter span (minimum, median, maximum, in billions, log scale) of the principal families in the inventory.

adopted by Mistral 7B and most later SLMs, divides the KV cache by the ratio of query to KV heads. Long context is reached by raising the rotary base or rescaling positions: Llama 3.2 uses a base of 500,000 to reach 128K tokens, Gemma 3 uses 1,000,000 on its global-attention layers, Phi-4 uses 250,000 with a 16K context, and Phi-3-mini's 128K checkpoint uses LongRoPE rescaling. Gated feed-forward layers are standard, SwiGLU in most families and GeGLU in Gemma.

Mixtures of experts. MoE layers [36, 119] activate a subset of expert feed-forward blocks per token through a learned router, so total and active parameters differ. Mixtral 8×7B [63] (46.7B total, 12.9B active) is the canonical example; DeepSeek-MoE [28] uses finer-grained and shared experts (about 2.8B active of 16B); Qwen3-30B-A3B [137] activates 3B of 30B; and Llama 4 Scout [92] activates 17B of 109B. All of these are small-active-parameter MoEs under C1 and C2, while Granite 4.0 Tiny (about 7B total, 1B active) [61] is an SLM. Apple's third-generation on-device model is a 20B-total sparse model that activates 1–4B parameters and keeps its full weights in flash, swapping experts into DRAM on demand [10]; its total count meets C2 at the bound, and its resident memory is set by the swapping policy rather than by the total.

Hybrid state-space models. Mamba [51] introduced hardware-aware selective state-space layers that process sequences in linear time. Jamba [78] interleaves Mamba and attention blocks (one attention layer in eight) and reports a KV cache eight times smaller than Mixtral's at 256K context.

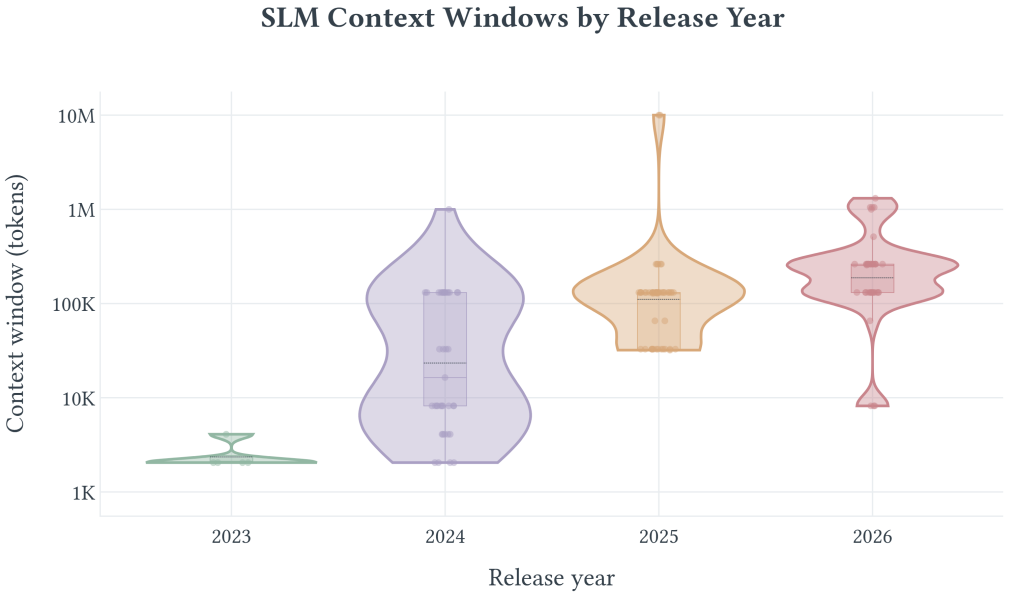


Fig. S4. Supported context window of inventory variants by release year (log scale), for the records that state one: 5 of 36 in 2023, 39 of 74 in 2024, 52 of 58 in 2025, and 39 of 41 in 2026. Each violin is a kernel-density estimate; the inner box marks the interquartile range, the dashed line the mean, and the dots individual models. Because 2023 coverage is thin and 2026 includes small-active-parameter MoEs, the trend is indicative.

Falcon-H1 [128] spans 0.5B–34B with parallel Mamba-2 and attention blocks; IBM's Granite 4.0 [61] interleaves Mamba-2 and attention at roughly 9:1 with a fine-grained MoE and reports over 70% memory reduction for long-context and multi-session inference relative to transformer-only models (C); NVIDIA's Nemotron Nano 2 [105] replaces most attention layers with Mamba-2 in a 9–12B model and reports up to 6× the throughput of Qwen3-8B on long reasoning traces (C); and the Qwen3.5 small series uses Gated DeltaNet layers with one full-attention layer in four [113]. Linear-time layers matter most where SLMs are deployed: long, streaming, memory-bound contexts on edge hardware.

S6.2 Training Strategy

Trained from scratch. Phi showed that "textbook-quality" synthetic and curated data yields large gains at fixed size [53, 74]: Phi-1 (1.3B) was trained on under 8B tokens yet reached competitive Python results. Gemma 3 [45] scales its token budget with size, from 2T tokens for the 1B variant to 14T for the 27B, under a curriculum that increases the share of mathematics, code, and multilingual data. Recurring practices are classifier-based quality filtering, synthetic data from larger teachers [34, 102], curricula that move toward harder data late in training, and continued pre-training for long context.

Distilled. Distillation [59] transfers capability from a teacher, through its internal representations (white-box) or its outputs and generated text (black-box). Orca [102] distilled GPT-4 explanation traces into a 13B student. The DeepSeek-R1 distillations [30] fine-tune Qwen2.5 and Llama students from 1.5B to 70B on long chain-of-thought traces from DeepSeek-R1, and Phi-4-mini-reasoning

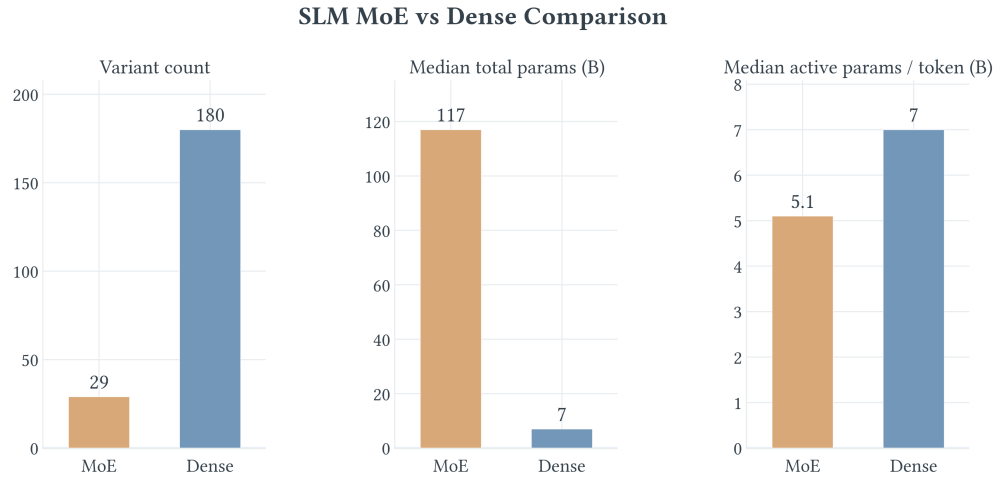


Fig. S5. Mixture-of-experts versus dense variants in the inventory: variant count (left), median total parameters (centre), and median *active* parameters per token (right). MoE models carry a much larger total parameter count at a per-token cost comparable to dense models, which is the distinction between conditions C1 and C2.

[134] combines distillation from a reasoning teacher with reinforcement learning at 3.8B. Gemma 3 and Gemma 4 [45, 46] distill every size from a larger teacher.

Pruned and distilled. NVIDIA's Minitron pipeline [103] prunes Nemotron-4 15B in depth and width, guided by activation-based importance, and retrains the pruned 8B and 4B students by distillation from the unpruned model, needing up to 40× fewer training tokens per derived model than training from scratch (a 1.8× compute saving for the family); a follow-up applies the recipe to Llama 3.1 8B and Mistral NeMo 12B [122]. Meta derived Llama 3.2 1B and 3B by pruning Llama 3.1 8B and distilling from the 8B and 70B models [91].

S6.3 Modality

Text-only. Most families ship text-only variants, and text-only models dominate the inventory (Figure S6).

Vision–language. PaliGemma (3B) [18] combines a SigLIP image encoder with a Gemma decoder and reports strong transfer after fine-tuning. Phi-3-vision (4.2B) [2] adds an image encoder to Phi-3-mini for document and chart understanding. Gemma 3 [45] attaches a SigLIP-based encoder, operating on 896 × 896 inputs with Pan-and-Scan cropping, to its 4B, 12B, and 27B variants; the 1B variant is text-only with a 32K context. Qwen2.5-VL [13] at 3B and 7B supports documents, charts, and video.

Audio and multimodal. Gemma 3n [48] nests a 2B-effective sub-model inside a 4B-effective one (MatFormer) and keeps per-layer embeddings off the accelerator, so that its 5B and 8B raw models run with the resident memory of roughly 2B and 4B models. Gemma 4 [46] spans 2.3B-effective to 31B with dense and MoE variants and a thinking mode. Voxtral Mini 3B (4.7B including its audio encoder) adds speech understanding and transcription to a 3B decoder under an open licence [80], and Apple's on-device models are multimodal [7, 10].

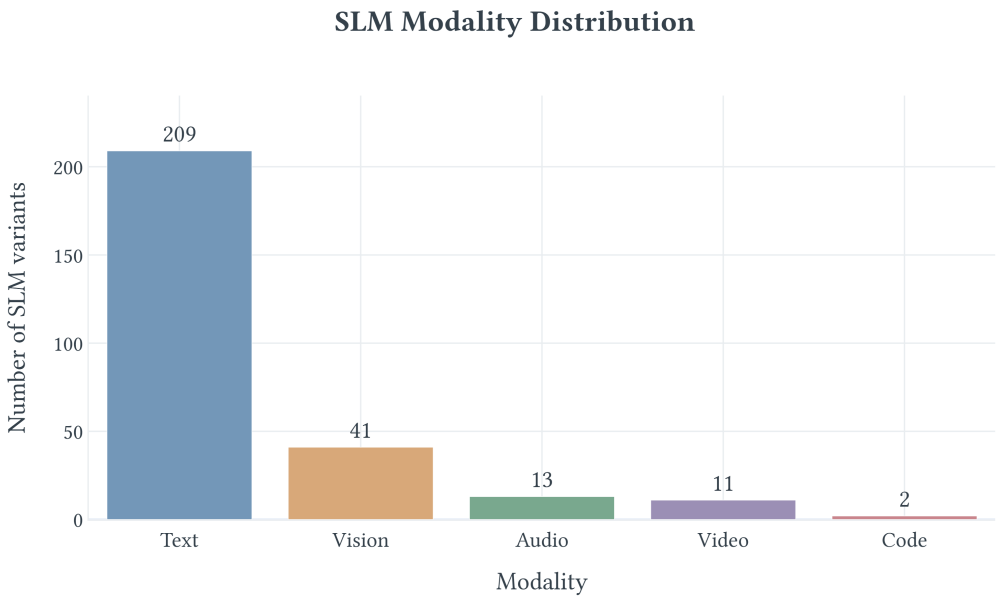


Fig. S6. Input modalities supported by inventory variants (a variant may support several). Text dominates, vision is common, and audio and video remain a small minority.

S6.4 Application Domains

Most SLMs are general-purpose models evaluated on broad benchmarks. Developer-trained domain models target narrow tasks:

Healthcare Meerkat-7B [68], fine-tuned on chain-of-thought traces synthesized from medical textbooks, reports 70.6% on MedQA, which its authors describe as the first 7B result above the roughly 60% USMLE pass threshold (C). OptimCLM [55] compresses clinical BERT encoders with under 5% macro-AUROC loss on outcome prediction (C).

Code Code Llama [117], StarCoder2 [83], and DeepSeek-Coder [54] are code-specialized families evaluated on HumanEval and MBPP (Supplementary Section S13.3).

Legal Small models fine-tuned on statutes, case law, and contracts target clause classification, obligation extraction, and regulatory triage. On-premises deployment can help meet privilege and data-residency obligations, subject to the scope conditions of Section 6.1; public evaluation of current small models in this domain is sparse (Supplementary Section S13.2).

Multilingual Aya 23 [11] allocates the capacity of an 8B decoder to 23 languages and reports outperforming the 13B Aya-101 [147] on the languages both cover (C); Aya-101 is an mT5-based encoder-decoder, outside C1. Qwen2.5-7B [112] documents coverage of Chinese, Japanese, Korean, Arabic, and the major European languages.

S7 Compression Methods in Detail

This section expands the summary of Section 4 of the main text.

S7.1 Post-Training Quantization

Post-training quantization needs only a small calibration set (typically 128–512 samples) and no gradient computation, which is why it is the dominant paradigm. GPTQ [39] quantizes layer by layer, using an approximation of the inverse Hessian of the layer-wise reconstruction loss to choose rounding decisions. activation-aware weight quantization (AWQ) [79] observes that about 1% of weights, those on large-magnitude activation channels, account for a disproportionate share of the quantization error, and applies per-channel scaling to protect them; its authors report higher accuracy than GPTQ at matched bit-width. At 8 bits the obstacle is activations rather than weights: LLM.int8() [31] keeps outlier dimensions in 16-bit, and SmoothQuant [133] migrates the difficulty from activations to weights to enable W8A8 integer kernels; both report accuracy essentially unchanged from FP16. SqueezeLLM [69] uses sensitivity-based non-uniform quantization with a dense-and-sparse decomposition. GGUF [47] packages weights, vocabulary, and metadata in one file, with block-wise "K-quant" schemes from Q2_K to Q8_0; mixed schemes such as Q4_K_M keep the most sensitive tensors at higher precision.

Two independent benchmarks bear on small models specifically. ThinkSLM [123] evaluates 72 small models, including quantized and pruned derivatives, on 17 reasoning benchmarks with three repeats, and finds that quantization preserves reasoning ability while pruning disrupts it. SLMQuant [129] evaluates quantization methods on two sub-1B models (SmolLM-135M and Qwen2.5-0.5B) on knowledge, commonsense, and language-modeling tasks, and finds that methods tuned on large models transfer imperfectly and that sensitivity depends on architecture: under W4A8 quantization the larger of its two models scored lower than the smaller, which the authors attribute to architectural differences rather than size. Neither study establishes that 4-bit loss grows monotonically as model size falls, and we make no such claim.

S7.2 Quantization-Aware and Low-Bit Training

Quantization-aware training simulates quantization in the training loop so that weights become robust to it. It recovers part of the accuracy that post-training quantization loses at 2–3 bits, with the margin widening as bit-width falls, but needs training data and compute, which is why it appears mainly in vendor pipelines such as Apple's 2-bit on-device quantization [7]. Its limit case is training in the low-bit format from the start: BitNet b1.58 [85] restricts weights to $\{-1, 0, 1\}$, and the released 2B4T model was trained in that format on four trillion tokens.

S7.3 Pruning

SparseGPT [38] extends the GPTQ machinery to weight removal and prunes OPT-175B and BLOOM-176B to 50–60% unstructured sparsity in one shot with a negligible increase in perplexity. Wanda [126] removes the weights with the smallest product of magnitude and input-activation norm, with no weight update, and its authors report parity with SparseGPT on LLaMA and Llama 2; that parity does not hold below 4B [146]. Structured pruning removes heads, channels, or layers and yields models that run on dense kernels: LLM-Pruner [86] removes coupled structures selected by gradient information and recovers quality with a short LoRA tuning; ShortGPT [88] deletes the layers that change their input least, and we found no study establishing how far this tolerance holds below 7B (D). Sheared-LLaMA [132] prunes Llama 2 7B to 1.3B and 2.7B and reports outperforming same-size open models at about 3% of the compute of training them from scratch (C).

S8 Knowledge Distillation Variants

S8.1 White-Box vs. Black-Box KD

The survey by Yang et al. [139] classifies LLM distillation into two paradigms:

White-box knowledge distillation (KD) has access to the teacher's intermediate representations. TinyBERT [64] pioneered layer-by-layer feature alignment, matching student hidden states, attention matrices, and embedding layers to teacher counterparts via MSE loss. In the decoder-only LLM setting, this requires pairing student and teacher layers, which is non-trivial when sizes differ substantially, and is computationally expensive but yields superior retention of teacher capabilities.

Black-box KD uses only the teacher's generated outputs, either token probabilities or free-form text, as the supervision signal. Orca [102] exemplifies text-output black-box KD: GPT-4 generates explanation traces and step-by-step reasoning, which a 13B LLaMA-based student learns to imitate. DeepSeek-R1 [30] extends this to long chain-of-thought reasoning distillation, fine-tuning 1.5B–70B Qwen and Llama students on the reasoning traces of its large model trained by reinforcement learning; its authors report that the reasoning patterns of the large model can be transferred to smaller ones in this way (C).

S8.2 Chain-of-Thought Distillation

A particularly productive form of black-box KD is chain of thought (CoT) distillation [130]: the teacher model generates explicit reasoning steps, and the student learns to produce or internalize these steps. Feng et al. [37] demonstrate counterfactual distillation, which augments CoT traces with contrastive incorrect reasoning, as a way to sharpen the student's reasoning boundaries. Liao et al. [77] propose Neural-Symbolic Collaborative Distillation, using a neuro-symbolic teacher to provide structured reasoning supervision that is more transferable to small students than purely neural CoT.

S8.3 Data Augmentation in KD

The survey by Xu et al. [136] highlights data augmentation (DA) as a central paradigm within KD frameworks. When the student lacks sufficient coverage of the teacher's knowledge space, the teacher can generate synthetic training instances targeting specific skill gaps, the mechanism underlying the Phi family's success [53]. Ballout et al. [15] use a roughly 3B teacher's most influential input tokens, identified by attribution methods such as saliency maps, as rationales for the student, and report improvements over standard fine-tuning and over prior distillation methods on four datasets.

S8.4 Self-Distillation

An emerging approach eliminates the external teacher entirely. Fu et al. [40] propose Dynamic Self-Distillation, in which the model uses its own predictions from previous mini-batches as soft targets during fine-tuning, providing a form of implicit regularization that reduces overfitting on small fine-tuning datasets, a critical challenge for SLMs.

S9 Audit Protocol Detail

This section expands the protocol summary of Section 5.1 of the main text.

S9.1 Benchmark Tasks and Metrics

The benchmarks that recur across the reviewed release documents span seven capability areas: MMLU [56] for broad knowledge; ARC-Easy and ARC-Challenge [26] and HellaSwag [142] for commonsense and language reasoning; GSM8K [27] and MATH [58] for mathematical reasoning; HumanEval [25] and MBPP [12] for code generation; TriviaQA [65] and SQuAD 2.0 [114] for question answering; MNLI [131] for natural language inference; and SST-2 [121] for sentiment classification. Each benchmark carries a standard metric: accuracy for multiple choice, exact match or F1 for question answering, pass@1 for code, exact match for GSM8K, and symbolic equivalence

for MATH. Instruction-following (MT-Bench-style [145]) and summarization are reported too sparsely and too heterogeneously to compare at all, and the conventions behind them are described in Supplementary Section S17.5. Section 5.3 takes up separately the question of whether any of these benchmarks bears on agentic suitability.

Five conditions determine what a benchmark number means, and a source that omits any of them has not reported a comparable result. *Benchmark version:* MMLU exists in the original and in MMLU-Pro, ARC in Easy and Challenge splits, GSM8K in original and GSM8K-Platinum revisions, and HumanEval in the original and HumanEval+ forms, and these are not interchangeable. *Prompt template:* whether the model's own chat template is applied, whether options are lettered or restated, and whether a system prompt is present. *Answer extraction:* whether a multiple-choice answer is scored by comparing per-option log-likelihoods or by parsing a generated letter, which can differ by several points on the same model and benchmark, and for chain-of-thought tasks, which span of the generation is taken as the answer. *Decoding settings:* greedy versus sampled, temperature, and for pass@k the number of samples. *Harness:* the framework and its version, most often `lm-evaluation-harness` [41] or HELM [76], each with its own defaults for all of the above.

The coverage of these five conditions across the reviewed sources is what Section 5.2 reports on, and it is poor. Which benchmarks a vendor reports, in which version, under how many shots, and with which extraction rule varies enough to prevent a common table.

S9.2 Coverage Ledger

Table S2 reproduces the coverage ledger summarized in Section 5.2. Its scope is the instruction-tuned checkpoints of the 14 cohort models from Microsoft, Alibaba, Meta, and Google, against the eight benchmarks that recur in their headline reporting; the other four cohort models (TinyLlama, SmolLM2, StableLM 2, Mistral 7B), each from a different developer, were not transcribed. Each cell was transcribed from the model's primary release document or official model card, listed with its table in the released CSV. Only coverage and stated protocol are recorded, not scores. Of the 112 cells, 69 carry an instruction-tuned value and 45 of those state a protocol; no benchmark is reported for all 14 models; and the largest group of models sharing one stated protocol on any benchmark is five (MMLU at 5 shots; GSM8K at 8 shots with chain-of-thought). Of the reconciliation rules written before collection, rule (a) (exclude a value that cannot be cross-checked against a second source) would exclude nearly every vendor cell by construction, so the finding is stated without it: protocol mismatch alone (rule (b)) and separate pre-trained and instruction-tuned tabulation (rule (c)) are sufficient to prevent a common table.

S9.3 Hardware and Quantization Scope

The review reads reported behavior against six reference configurations spanning the cloud-to-edge spectrum (Table S5) and three precisions: FP16 (reference), INT8, and INT4, with GGUF Q4_K_M, GPTQ group-128, and AWQ group-128 treated as distinct INT4 subclasses when the source distinguishes them. Inference-framework attribution (vLLM [70], llama.cpp [47], ONNX Runtime [93], ExecuTorch [89], MLC-LLM [100]) follows the stack used by the source of each figure; cross-framework comparisons are made only when compared figures share a framework or are explicitly flagged.

S9.4 Limitations of the Evidence Base

Because the figures are compiled from heterogeneous public sources rather than a single controlled pipeline, four limitations bound the strength of every comparison below. **Prompt-format**

Table S2. Coverage ledger for the instruction-tuned checkpoints of the 14 cohort models from the four largest vendors: the protocol under which each model's primary release document reports each benchmark. Numbers are shot counts; "c" marks chain-of-thought; "SE" OpenAI simple-evals defaults; "u" a value reported without a stated protocol; "–" not reported. Sources and locations are listed in `audit_coverage_ledger.csv`; transcribed on 8 October 2026.

Model	MMLU	MMLU-Pro	ARC-C	HellaSwag	GSM8K	MATH	HumanEval	MBPP
Phi-3-mini	5	–	10	5	8c	–	0	3
Phi-3.5-mini	5	0c	10	5	8c	0c	0	3
Phi-4-mini	5	0c	10	5	8c	0c	–	–
Phi-4	SE	–	–	–	–	SE	SE	–
Qwen2.5-0.5B	–	u	–	–	u	u	u	u
Qwen2.5-1.5B	–	u	–	–	u	u	u	u
Qwen2.5-3B	–	u	–	–	u	u	u	u
Qwen2.5-7B	–	u	–	–	u	u	u	u
Llama 3.2-1B	5	–	0	0	8c	0c	–	–
Llama 3.2-3B	5	–	0	0	8c	0c	–	–
Gemma 2-2B	u	–	–	–	–	–	–	u
Gemma 2-9B	u	–	–	–	–	–	–	u
Gemma 3-1B	0	0	–	–	0c	0	0	3
Gemma 3-4B	0	0	–	–	0c	0	0	3
Reported (of 14)	10	8	5	5	11	11	9	10

and harness variation: different sources use different templates and answer-extraction heuristics, which can shift reported accuracy by several points under the same nominal benchmark. **Self-reporting bias:** vendor-reported scores are systematically more favorable than third-party reproductions; where third-party figures exist, comparisons preferentially use them and flag the divergence. **Condition-coverage sparsity:** not every model has reported scores for every task, tier, or precision. Missing values are never imputed, interpolated, or carried across from a sibling model size or from a base checkpoint to its instruction-tuned variant; a tuple with no qualifying source is recorded as absent, and absence is itself reported, since the pattern of what vendors decline to publish is part of the finding of Section 5.2. No statistic in this paper is computed over a set from which absent tuples were silently dropped. **Snapshot staleness:** the ecosystem evolves monthly, and figures reflect the state of public reporting at the collection window. These limitations motivate the empirical-verification agenda of Section 9.3.

S9.5 Quantization Impact

The reported picture for the sub-10B regime is summarized in Table 3 of the main text. INT8 post-training quantization is reported as effectively lossless by the authors of the main methods [31, 133] (C). At 4 bits, an independent benchmark of 72 small models finds reasoning largely preserved [123], and an independent benchmark of two sub-1B models finds that methods tuned on large models transfer imperfectly, with sensitivity depending on architecture [129] (B); neither establishes that the loss grows monotonically as size falls. AWQ is reported to retain more accuracy than GPTQ at matched bit-width [79] (C), and mixed schemes such as GGUF Q4_K_M keep the most sensitive tensors at higher precision [47]. In the controlled comparisons at 0.5–3.8B, post-training quantization preserved fidelity better than post-training pruning [123, 146] (A); Section 4.2 gives the scope.

Per-model, per-benchmark degradation figures are not reported consistently enough across this cohort to tabulate: sources differ in harness, calibration set, and group size, and few publish a

matched FP16 baseline alongside the quantized result. Measuring them under a single toolchain is one of the verification studies proposed in Section 9.3.

The memory consequence, by contrast, is firmly established, because it follows from the storage format rather than from measurement. Table S3 gives weight footprints estimated from parameter count and effective bits per weight. The FP16 column is exactly two bytes per parameter. The four-bit columns correspond to 4.4–5.8 effective bits per parameter: four-bit weights plus per-group scales and zero-points, with the embedding and output layers typically kept at higher precision, which is a larger share of a smaller model. The reduction therefore falls short of the nominal 4×, at 2.8 to 3.6× across the table, and is smallest for the smallest models. Qwen2.5-7B drops from 15.2 GB to 4.2–4.5 GB, and Phi-4-mini (3.8B) from 7.6 GB to 2.2–2.4 GB, which is what moves each of them across a hardware tier boundary. The exact figure for a given artifact is its file size, which should be read from the artifact rather than from this table.

Table S3. Model-weight memory footprint by precision (GB). FP16 is two bytes per parameter; the four-bit columns are estimates at 4.4–5.8 effective bits per parameter, including the group scales and higher-precision tensors each scheme retains, which is why the reduction (2.8–3.6×) is below the nominal 4×. Activation and KV-cache memory are additional and depend on context length and batch size.

Model	FP16	GPTQ-4bit	AWQ-4bit	Q4_K_M
Qwen2.5-7B	15.2	4.2	4.4	4.5
Phi-4-mini	7.6	2.2	2.3	2.4
Llama 3.2-3B	6.4	1.9	2.0	2.1
SmolLM2-1.7B	3.4	1.1	1.1	1.2
TinyLlama-1.1B	2.2	0.7	0.8	0.8

S10 Cohort and Hardware Reference Tables

Table S4 gives the architectural specifications of the 18-model cohort of Section 5; Tables S5 and S6 give the reference hardware of Sections 5 and 6.

Table S7 summarizes the four primary facets of the taxonomy of Section 3 with representative models.

Table S8 compares the deployment frameworks of Section 6.1.1.

Table S9 gives the secondary facets of the taxonomy.

S11 Platforms and Device Tiers in Detail

S11.1 Apple Intelligence (AFM)

Apple's Foundation Models [7, 8] are the most fully documented on-device SLM deployment. The 2025 report [7] describes a ∼3B on-device model, optimized for Apple silicon through KV-cache sharing (dividing the model so the second block reuses the first block's cache, a reported 37.5% memory saving) and 2-bit quantization-aware training, paired with a Parallel-Track Mixture-of-Experts server model on Private Cloud Compute. The on-device model is reported competitive with Qwen3-4B and Gemma-3-4B on its target tasks, and the server model with Llama 4 Scout (C). The third-generation family [10], five models built in collaboration with Google, adds a 20B-total on-device model that activates 1–4B parameters per token and keeps its full weights in flash, swapping experts into DRAM as needed; Private Cloud Compute was extended to NVIDIA GPUs in Google Cloud. This followed a January 2026 joint statement that Apple's next foundation models would be based on Gemini models and cloud technology [49]. Google's Gemini Nano, the on-device member of the Gemini family [42], occupies the same design point on Android.

Table S4. Architectural specifications of reviewed models (as reported by each model's official release materials).

Model	Params	Layers	Hidden	Attention	Context	Vocab	Train Tokens
TinyLlama-1.1B	1.1B	22	2,048	GQA-32/4	2K	32K	3.0T
SmolLM2-1.7B	1.7B	24	2,048	MHA-32h	8K	49K	11.0T
StableLM 2-1.6B	1.6B	24	2,048	MHA-32h	4K	100K	2.0T
Qwen2.5-0.5B	0.5B	24	896	GQA-14/2	32K	152K	18.0T
Qwen2.5-1.5B	1.5B	28	1,536	GQA-12/2	32K	152K	18.0T
Qwen2.5-3B	3.0B	36	2,048	GQA-16/2	32K	152K	18.0T
Qwen2.5-7B	7.6B	28	3,584	GQA-28/4	128K	152K	18.0T
Llama 3.2-1B	1.2B	16	2,048	GQA-32/8	128K	128K	9.0T
Llama 3.2-3B	3.2B	28	3,072	GQA-24/8	128K	128K	9.0T
Gemma 2-2B	2.6B	26	2,304	GQA-8/4	8K	256K	2.0T
Gemma 2-9B	9.2B	42	3,584	GQA-16/8	8K	256K	8.0T
Gemma 3-1B	1.0B	26	1,152	GQA-4/1, LG	32K	262K	2.0T
Gemma 3-4B	4.3B	34	2,560	GQA-LG	128K	262K	4.0T
Mistral 7B	7.3B	32	4,096	GQA-32/8	8K	32K	n/a
Phi-3-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.3T
Phi-3.5-mini	3.8B	32	3,072	MHA-32h	128K	32K	3.4T
Phi-4-mini	3.8B	32	3,072	GQA-24/8	128K	200K	5.0T
Phi-4	14.0B	40	5,120	GQA	16K	100K	~10T

Note: MHA = Multi-Head Attention; GQA = Grouped-Query Attention (Q/KV heads); GQA-LG = GQA with local–global interleaving. Every model in this cohort is dense, so total and active parameters coincide and a single parameter column suffices; no small-active-parameter MoE model (Section 1.2.2) is in the cohort, because none has public reporting complete enough to include.
On training-token counts. These are the least reliable column in the table, and their status differs by row. Marked *per-variant* where the source states a figure for that specific size: Gemma 2-2B (2.0T), Gemma 2-9B (8.0T), Gemma 3-1B (2.0T), Gemma 3-4B (4.0T), Phi-3-mini (3.3T), Phi-3.5-mini (3.4T), Phi-4-mini (5.0T), TinyLlama (3.0T), SmolLM2-1.7B (11.0T), StableLM 2 (2.0T). Marked *series-level* where the source reports one figure for the whole family and the individual sizes are not separately stated: Qwen2.5 (18.0T across 0.5B–7B) and Llama 3.2 (9.0T across 1B and 3B). Phi-4's ~10T is an approximation given as such by its source. Mistral 7B's training-token count is not disclosed. We report series-level figures in the size rows they cover rather than omitting them, but they should not be read as independent per-model measurements, and no claim in this paper relies on comparing a series-level figure against a per-variant one.
On context length. The value is the maximum the primary source documents, which for several models is reached through position interpolation at inference rather than trained natively: Qwen2.5-7B is natively 32K and extended to 128K, and Phi-3-mini's 128K is a separate long-context checkpoint from the 4K base.

S11.2 Samsung Gauss

Samsung announced Gauss in November 2023 as a family of generative models in three parts, Gauss Language, Gauss Code, and Gauss Image, used first for internal productivity and planned for its products [87]. Samsung has published no parameter count, architecture, or on-device specification for it, so Gauss fails criterion M1, is not in the inventory, and we make no claim about its deployment. It is listed here because it is frequently cited as an on-device SLM, and the published record does not support that description.

S11.3 General Mobile Deployment Considerations

SLMs targeting mobile deployment must navigate three constraints. (1) *Memory*: Consumer smartphones typically provide 6–12 GB RAM, of which the OS uses 2–4 GB, leaving 4–8 GB for applications. A 3B model at Q4 requires ~2 GB, comfortable for most flagship devices. (2) *Thermal*:

Table S5. Reference hardware configurations used to organize the deployment discussion of Section 6. They define the tiers against which reported figures are read; no per-model, per-configuration result is tabulated, because the sources do not report one consistently (Section 6.2).

Configuration	Hardware	RAM/VRAM	TDP	Use Case
Cloud GPU	NVIDIA A100 80GB	80 GB HBM2e	300W	Training + serving
High-end Consumer	NVIDIA RTX 4090	24 GB GDDR6X	450W	Local inference
CPU/iGPU Laptop	AMD Ryzen 7 7840HS	32 GB DDR5	35–54W	Laptop deployment
Edge GPU Module	NVIDIA Jetson Orin Nano	8 GB LPDDR5	7–15W (25W Super)	Edge AI / robotics
Edge SBC	Raspberry Pi 5	8 GB LPDDR4X	12W	Embedded IoT
Mobile SoC	Snapdragon 8 Gen 3	12 GB LPDDR5X	~8W sustained	Smartphone

Table S6. Detailed specifications of the devices referenced in this section.

Specification	Raspberry Pi 5	Jetson Orin Nano	Galaxy S24+	Laptop
CPU	BCM2712 4×A76 @ 2.4GHz	6×A78AE @ 1.5GHz	Snapdragon 8 Gen 3	Ryzen 7 7840HS
GPU	VideoCore VII	1024 CUDA (Ampere)	Adreno 750	Radeon 780M
NPU	None	None	Hexagon	Ryzen AI
RAM	8 GB LPDDR4X	8 GB LPDDR5 (unified)	12 GB LPDDR5X	32 GB DDR5-5600
Memory bandwidth	~17 GB/s	68 GB/s	76.8 GB/s	89.6 GB/s
Storage	microSD / NVMe	NVMe	UFS 4.0	NVMe SSD
TDP	12W	7–15W (25W Super)	~8W sustained	35–54W
Inference engine	llama.cpp	MLC LLM	ExecuTorch	llama.cpp

Table S7. Summary of the four primary facets of the SLM taxonomy with representative models. Assignment is multi-label: a model may carry several categories on one facet, and the same model may therefore appear more than once in a block.

Facet	Category	Key Characteristics	Representative Models
Architecture	Dense Transformer	All params active per token; GQA, RoPE, SwiGLU	Phi-4-mini, Qwen3, SmolLM3, Gemma 3
	MoE	Sparse activation; total ≫ active params	Granite 4.0 Tiny (SLM); Mixtral 8×7B, Qwen3-30B-A3B (small-active MoE)
	Hybrid SSM-Transformer	Linear-time; interleaved SSM/attention blocks	Falcon-H1, Granite 4.0, Nemotron Nano 2
Training Strategy	From scratch	Curated/synthetic data; full pre-training	Phi, Qwen2.5/Qwen3, SmolLM3
	Distillation	Knowledge transfer from larger teacher	Gemma 3/4, DeepSeek-R1-Distill, Phi-4-mini-reasoning
	Pruning-based	Parameter removal from large model + distillation	Minitron, Sheared-LLaMA
Modality	Text-only	Tokenized text I/O; majority of production SLMs	Mistral 7B, SmolLM3, Qwen3
	Vision-Language	Image + text; CLIP/SigLIP encoder added	Phi-3-vision, Gemma 3 (4B and above), PaliGemma
	Audio/Multimodal	Text + vision + audio in one sub-10B model	Gemma 3n, Gemma 4, Apple AFM
Domain	General-purpose	Multi-domain training; evaluated on broad benchmarks	Phi, Gemma, Llama, Qwen, Mistral
	Healthcare	Clinical NLP; on-premises for data residency	Meerkat 7B, OptimCLM
	Code	Code corpora; FIM training; HumanEval/MBPP focus	Code Llama, StarCoder2, DeepSeek-Coder
	Legal/Multilingual	Domain corpora; regulatory and cross-lingual tasks	Aya 23 8B, Qwen2.5 multilingual

Sustained inference on mobile NPUs is throttled by thermals; realistic sustained throughput is lower than peak throughput benchmarks suggest. (3) *Battery*: published per-device battery figures are scarce and not reported under a common protocol. The fundamental constraint in edge decoding is memory bandwidth rather than compute: flagship phones and mainstream laptops offer on the order of 50–100 GB/s (single-board computers well under 30 GB/s, unified-memory laptop designs considerably more) versus 2–3 TB/s for datacenter GPUs [135], making INT4 quantization not merely a storage optimization but a performance enabler for single-stream generation.

Table S8. Key SLM deployment frameworks, their primary targets, and characteristics. Throughput is not tabulated, because project-reported figures are measured on different hardware and settings and are not comparable across rows (Section 6.2).

Framework	Primary Target	Quantization	Notes
llama.cpp [47]	CPU, Metal, CUDA, ROCm	GGUF (Q2–Q8)	Widely used reference runtime; minimal dependencies; ecosystem includes Ollama, llamafile
ONNX Runtime [93]	GPU, CPU, NPU, WASM	INT4/INT8	Cross-platform; DirectML, CoreML, CUDA backends
MLX [9]	Apple Silicon	4-bit	Unified memory; no host–device weight transfers
TensorRT-LLM [106]	NVIDIA GPUs	INT4/INT8/FP16	Continuous batching; kernel fusion
ExecuTorch [89]	Mobile (iOS, Android)	INT4	Small base runtime; CoreML, QNN, ARM, MediaTek, Vulkan backends; used for Meta's Llama 3.2 on-device measurements
MLC-LLM [100]	iOS, Android, web, embedded GPU	4-bit	TVM compilation; WebGPU support
WebLLM [101]	Web browsers	4-bit WASM	WebGPU acceleration; OpenAI-compatible API

Table S9. Secondary facets. These do not describe what a model is but whether it can be adopted, and they are the facets practitioners report finding hardest to recover from model cards. Access and license are recorded per variant in the released inventory, though for 96 of the 209 records the license is given only as a pointer to the source; the other facets are assigned in this paper to representative models.

Facet	Values	Why it gates adoption
Licensing	Permissive (Apache-2.0, MIT); custom vendor licence with use restrictions; research-only; unspecified; proprietary/API-only	Determines commercial use, redistribution of fine-tunes, and whether an on-premises deployment is legal at all
Data transparency	Full (corpus released); partial (composition described, data not released); disclosed sources only; undisclosed	Determines whether contamination, licensing of training data, and domain fit can be assessed
Safety alignment	Preference-optimized with a published safety evaluation; instruction-tuned without one; base model only	A base checkpoint wired into an agent with tool access has no refusal behavior to rely on
Reasoning mode	None; always-on extended reasoning; controllable budget; separate reasoning checkpoint	Determines token cost per call and whether latency is predictable, which is decisive inside an agent loop
Deployment target	Documented inference engine(s) and hardware class the vendor ships an artifact for	An unquantized checkpoint with no GGUF, ONNX, or ExecuTorch export is not an edge model whatever its size

S11.4 Edge and IoT Deployment

Below smartphone-class hardware lies a diverse ecosystem of edge devices: Raspberry Pi single-board computers, NVIDIA Jetson modules, and microcontrollers. SLMs targeting this tier typically require:

Sub-1B or extreme quantization A Raspberry Pi 5 (8 GB RAM, Cortex-A76 CPU, about 17 GB/s of memory bandwidth) can hold 4-bit models up to a few billion parameters, and its decode rate is bounded by that bandwidth. A benchmark of runtimes on single-board computers reports that llamafile reaches up to 4× the throughput of Ollama at 30–40% lower power for equivalent models [104] (B).

Jetson Orin NVIDIA's Jetson Orin modules, with TensorRT-LLM [106] among their runtimes, hold quantized 3–7B models in 8–64 GB of unified memory, with the larger-memory NX and AGX parts suited to the upper end of that range, for industrial and robotics use.

Microcontrollers (MCU) ARM Cortex-M class microcontrollers carry roughly 512 KB–2 MB of SRAM. Even a 10M-parameter model needs about 5 MB at 4 bits, and a 100M-parameter one about 50 MB, so a language model of the size this survey covers cannot be resident in microcontroller SRAM, and would at best stream weights from external memory at a large cost in speed. Keyword detection and sensor analytics on these parts are served by task-specific networks far smaller than any model in the inventory; we found no documented SLM deployment at this tier, and it is outside the scope of this paper.

Table S6 details the specifications of the four representative devices referenced throughout this section, from the Raspberry Pi 5 up to a laptop-class CPU.

S12 Conversion Pipeline and Barriers to Adoption

S12.1 Converting an LLM Agent to an SLM Agent

For teams with an existing LLM-only agent, Belcak et al. [17] propose a six-step conversion pipeline, which we summarize here because it operationalizes the position; the steps are theirs. **S1, instrumentation:** log every non-interface model call (inputs, outputs, latency) from the running agent, with encryption and access control. **S2, curation:** strip personally identifiable and protected health information and application secrets from the logs. **S3, clustering:** apply unsupervised clustering to the curated calls to surface the recurring request types, which are the specialization targets. **S4, selection:** for each cluster, choose a candidate SLM by matching required capability against benchmarks, license, and deployment constraints. **S5, specialization:** fine-tune the chosen SLM on the cluster's curated data, typically with LoRA or QLoRA. **S6, iteration:** re-run the loop periodically as the traffic distribution drifts. Two additions follow from H. Step S3 is where the conditions are assessed: each cluster's output schema (H1), input length (H3), and the availability of validity and correctness checks (H4a, H4b) can be checked by inspection, and its volume can be compared with a pilot estimate of the break-even N^* (H2), recorded before the comparison; clusters that fail are left with the generalist. And each converted cluster should be evaluated on cost per completed task against the unconverted agent, which is the measurement the literature lacks.

S12.2 Barriers to Adoption

Belcak et al. [17] identify three reasons the shift has lagged the technical case. First, large centralized-inference investments create sunk-cost inertia and an operational bias toward one big model behind one API (B1). Second, SLMs are still evaluated on generalist leaderboards (MMLU, broad chat benchmarks) that are misaligned with agentic utility, so their fitness for delegated sub-tasks is under-measured (B2). Section 5 shows that the generalist record is also too inconsistent across vendors to support cross-model comparison, so it cannot be used to choose among small models even for general capability. Third, SLM releases receive far less attention than frontier launches, so awareness lags capability (B3). The first and third are not technical obstacles. The second is different, and Section 7.3 shows it to be partly a real evidential gap rather than only a misdirected emphasis: agent-specific evaluation exists and is maturing [16, 67, 81, 108, 141], and developer reports support per-call competence for specialized small models, but independent multi-step evaluations of current sub-10B models are sparse, and no study compares cost per completed task for small-model-first and large-model agents. Until that changes, the strength of the case for hypothesis H will remain uneven in the way Table 5 records.

S12.3 Learned Routing and Cascades

Three designs recur. *Cascades* try the cheap model first and escalate on a quality signal: FrugalGPT [24] learns which sequence of LLM APIs to query, and AutoMix [4] escalates when the small model's

own few-shot self-verification judges its answer unreliable, which is the pattern H prescribes when the checker of H4 is the model itself. *Predictive routers* decide before generation: Hybrid LLM [33] routes by predicted query difficulty with a quality level tunable at test time, and RouteLLM [107] trains a lightweight router on human-preference data to predict whether a weak model's answer would be judged as good as a strong model's, reporting transfer of the trained router to unseen strong/weak model pairs. *Decomposing routers* push the decision below the query level: Division-of-Thoughts [118] decomposes a task into sub-questions and assigns each to a local SLM or a cloud LLM according to estimated difficulty, for on-device agents. The open problems are difficulty estimation without an oracle, routing objectives that jointly weigh accuracy, latency, privacy sensitivity, and network availability, and handoff that does not produce a visible seam in output quality [84].

A different form of SLM–LLM collaboration leaves the large model in charge of every output and uses the small one only to make it faster. In speculative decoding a small draft model proposes several tokens and the large model verifies them in one forward pass, accepting a prefix, so that the output distribution is that of the large model [23, 72]; Medusa [21] and EAGLE [75] replace the separate draft model with lightweight heads on the large model itself. This is not delegation in the sense of H, since no call's answer comes from the small model, but it is the most widely deployed way in which small models already reduce the cost of large-model inference, and it is complementary to routing: an escalated call can itself be served speculatively.

S13 Domain-Specific Applications in Detail

This section expands the summary of Section 8.3 of the main text.

S13.1 Healthcare and Clinical NLP

Healthcare natural language processing (NLP) presents demanding requirements: high factual accuracy, interpretability, compatibility with clinical workflows, and operation within a regulatory regime that constrains where identifiable patient data may be processed. That regime is the pull toward on-premises models, and it needs stating precisely, because it is frequently overstated.

On-premises deployment is not compliance. Neither HIPAA nor the GDPR is satisfied by the location of a model's weights. HIPAA compliance is a property of a covered entity's administrative, physical, and technical safeguards taken together: access control, audit logging, integrity controls, transmission security, workforce training, breach notification, and business-associate agreements where a third party is involved. The GDPR requires a lawful basis for processing, purpose limitation, data minimization, a data protection impact assessment for high-risk processing, and, for automated decisions with legal or similarly significant effects, the safeguards of Article 22. A locally hosted model can fail every one of these. What on-premises deployment does is remove one specific difficulty, the transfer of identifiable data to a processor outside the entity's control, which under HIPAA otherwise requires a business-associate agreement and under the GDPR raises transfer and sub-processor questions. That is a real and often decisive simplification, and it is the reason the pull exists. It is not a compliance property of the model, and no claim in this paper should be read as asserting one. The same scope conditions as Section 6.1 apply: an on-premises model whose application logs prompts to a cloud service has reintroduced the transfer it was chosen to avoid.

The Meerkat family [68] demonstrates the potential of domain-specific SLM fine-tuning. Built on a 7B base with instruction fine-tuning on curated clinical QA data and chain-of-thought traces synthesized from medical textbooks, Meerkat-7B reports 70.6% on MedQA (USMLE-style), which its authors describe as the first 7B result above the USMLE passing threshold, ahead of earlier open medical models and GPT-3.5 across six medical benchmarks though below GPT-4 (C). The larger Meerkat-70B is reported to exceed GPT-4 by 1.3 points on average and to diagnose 21 of 38 complex

clinical cases, against 21.8 for GPT-4 and 13.8 for human respondents (C). Meerkat-7B runs on a single GPU; Meerkat-70B needs about 140 GB at 16-bit precision.

OptimCLM [55] distils an ensemble of domain-adapted clinical BERT encoders into the smaller BERT-PKD and TinyBERT, then applies unstructured pruning and 8-bit quantization, reporting up to 22.88× compression with under 5% loss in macro-AUROC on length-of-stay, mortality, diagnosis, and procedure prediction from MIMIC-III admission notes. It shows that the compression stages compose for clinical classifiers that can run on hospital workstations; it is not evidence about generative SLMs.

The broader SLM healthcare ecosystem includes domain-pretrained biomedical models in the 2–3B range, clinical note generation models, and radiology-specific models. A key trend is the use of retrieval-augmented generation (RAG) [73] to anchor SLM outputs to verified clinical knowledge bases, reducing hallucination rates in patient-facing applications.

S13.1.1 The State of the Evidence in This Domain. Healthcare is the best-evidenced of the verticals surveyed here, and it is still weaker evidence than it appears. Three qualifications apply to everything above.

Licensing-exam benchmarks are not deployment evidence. MedQA and its siblings are multiple-choice examinations designed to test candidate physicians, and performance on them establishes that a model can answer exam questions. It does not establish accuracy on the tasks clinical systems actually perform: coding an encounter, drafting a discharge summary, extracting a medication list from a note, or flagging a contraindication. The Meerkat results are grade-C, author-reported on public benchmarks with a documented method, which is a good standard for the model literature and not the standard a clinical deployment decision requires.

Independent prospective evaluation is largely absent. We found no prospective study evaluating a sub-10B model in a live clinical workflow with outcome measurement, and no head-to-head comparison of an on-premises small model against a cloud frontier model on the same institution's data under the same protocol. Retrospective benchmark performance and deployment performance are known to diverge in clinical NLP, and nothing in the current SLM literature closes that gap.

Compression results are single-study. The OptimCLM finding [55] that compressed clinical encoders lose under 5% in macro-AUROC on outcome prediction is one pipeline, on one dataset, for classification rather than generation. It has not been independently reproduced, and no comparable study exists for a compressed generative SLM on clinical tasks such as coding an encounter or drafting a discharge summary.

The honest summary is that healthcare provides the strongest published case for domain-specific small models and that the case rests on exam benchmarks and single studies rather than on deployment evidence. Establishing per-task clinical benchmarks with independent evaluation, and reporting per-group performance floors of the kind Section S14.4 argues for, is an open need in this vertical specifically.

S13.2 Legal NLP

Legal document processing, which covers contract review, compliance monitoring, regulatory change analysis, and case law search, involves long and precisely structured documents where hallucination is particularly dangerous. SLMs address legal NLP deployment constraints in two ways: (1) by enabling on-premises deployment to satisfy attorney-client privilege and data security requirements; and (2) by enabling cost-effective processing of the massive volumes of routine legal work that would be prohibitively expensive with frontier LLMs.

The argument for domain-adapted SLMs (3B–7B, fine-tuned on contract corpora) in this vertical is structural rather than empirical: contract review, compliance triage, and regulatory monitoring

consist of bounded, highly repetitive classification and extraction operations over documents that frequently cannot leave the client's infrastructure, which is precisely the profile the conversion pipeline of Section S12.1 targets. Narrow fine-tuning on a specific regulatory regime concentrates in-domain knowledge that a generalist model must retrieve or infer.

We state the evidential position for this vertical explicitly, because it is weaker than for the other three and it would be easy to read the structural argument above as though it were a result. Public legal benchmarks exist: LegalBench [52] collects 162 legal-reasoning tasks, LexGLUE [22] groups legal language-understanding tasks, and CUAD [57] provides expert-annotated contract clauses. What is missing is their use for the current generation of small decoder models: we found no evaluation of 2024–2026 sub-10B models across them under a common protocol, and deployment comparisons rest on vendor pilot reports that are not independently verifiable, often because the corpora are client-confidential. Two structural features of the domain make this worse: an error in contract or compliance work can be both consequential and hard to detect, which is exactly the condition under which hypothesis H's verifiability requirement (Section 1.3) fails; and long, cross-referential documents strain the short-context condition. Legal work is therefore a domain where the economic case for small models is strong and the capability evidence is absent, and we present the section as such rather than as a claim of parity. Establishing open legal-domain benchmarks with independent evaluation is a prerequisite for saying anything stronger.

S13.3 Code Generation

Code-focused SLMs have matured into a category distinct from general SLMs, with specialized pretraining corpora (GitHub, Stack Overflow, documentation), code-aware tokenization, and evaluation on function-level benchmarks (HumanEval, MBPP, SWE-Bench).

Code Llama [117] Meta's family of code models fine-tuned on Llama 2 (7B, 13B, 34B). Code Llama 7B achieves 33.5% HumanEval pass@1.

StarCoder2 [83] BigCode's 3B/7B/15B models trained on The Stack v2 (600+ programming languages). StarCoder2-7B reports 35.4% HumanEval pass@1, and StarCoder2-15B 46.3%.

DeepSeek-Coder [54] A 1.3B–33B family trained from scratch on 2T tokens. The 6.7B base model reports 49.4% on Python HumanEval, and its instruction-tuned version 78.6%, above the 76.2% the same report gives for GPT-3.5-Turbo.

Codestral [98] Mistral's 22B code model trained on 80+ programming languages with fill-in-the-middle (FIM) training for IDE integration. At 22B dense it is just outside the C1/C2 bound and is not in the inventory; it is listed as the upper reference for this category.

S13.4 Education and Multilingual NLP

Multilingual SLMs address the linguistic coverage gap in AI assistants: the majority of LLM pretraining data is English, leaving speakers of lower-resource languages with markedly worse performance. Aya 23 [11] allocates the capacity of an 8B decoder to 23 languages and reports outperforming its predecessor Aya-101 [147], a 13B mT5-based encoder-decoder covering 101 languages (outside C1), on the languages both cover (C); the pair illustrates the breadth-versus-depth trade-off at this size. Qwen2.5 [112] and Gemma 3 [45] document large non-English pretraining mixtures and report multilingual evaluations (C). For educational use, which includes on-device tutoring and low-connectivity tools, the requirements beyond language coverage are low hallucination rates and age-appropriate safety filtering, neither of which is reported comparably across small models.

S14 Model-Level Challenges in Detail

S14.1 Hallucination Mitigation in Small Models

Hallucination, the confident generation of factually incorrect content, is an intrinsic failure mode of autoregressive language models, and it is more pronounced in smaller models [144]. The core issue is information capacity: a 3B model's parameters can encode far fewer facts than a 175B model, and when queried about knowledge at the boundary of its training distribution, small models are more likely to confabulate plausible-sounding but incorrect responses.

Mitigation strategies include: (1) *RAG* [73], which anchors SLM generation to retrieved documents rather than parametric memory; (2) *uncertainty quantification* through sampling and ensemble methods; (3) *RLHF with truthfulness rewards* to penalize confident incorrect responses; and (4) *calibration post-processing* to align model confidence to actual accuracy.

S14.2 Reasoning Limitations Under Parameter Constraints

Language-modeling loss falls as a power law in parameter count [66], and in the reported results reviewed here the gap between small and large models is widest on multi-step reasoning, particularly in mathematics, formal logic, and scientific inference (D). At the 1B–7B scale, SLMs struggle with problems that require holding many intermediate states simultaneously or with deep deductive chains.

Emerging directions include: (1) *Test-time scaling*: allocating additional inference compute (more generation tokens, repeated sampling, tree-of-thought search) to substitute for model capacity; the DeepSeek-R1 authors [30] report that distilling the long chains of thought of their large reasoning model into 1.5B–70B students transfers much of its mathematical-reasoning ability (C). (2) *Neuro-symbolic integration*: augmenting SLMs with symbolic solvers (SAT solvers, theorem provers, calculators) to handle formal reasoning externally. (3) *Speculative decoding* [23, 72] using a small draft SLM to propose tokens verified by a larger model, which speeds up the larger model rather than substituting for it (Section S12.3).

S14.3 Hardware Co-Design and NPU Optimization

The efficiency of SLM inference depends as much on hardware as on model architecture. Modern consumer hardware increasingly includes dedicated AI accelerators: Apple ANE, Qualcomm Hexagon NPU [111], Samsung Exynos NPU, and Intel NPU. These accelerators are optimized for specific operation types (INT4/INT8 matrix multiplication) and data layouts.

A critical open problem is *hardware-model co-design*: developing SLM architectures whose attention patterns, FFN widths, and layer counts align with the data bus widths and tile sizes of target NPUs. Future SLMs trained with hardware constraints as first-class objectives, rather than retrofitting quantization after the fact, will likely yield significant efficiency improvements.

S14.4 Ethical Considerations: Bias Amplification in Compressed Models

Model compression introduces an underexplored ethical risk: *bias amplification*. Pruning and quantization may disproportionately affect model performance on underrepresented demographic groups, dialects, and languages, as the model's remaining capacity may preferentially serve the majority distributions present in training data [20]. Zhou et al. [146] find that quantization preserves multilingual perplexity better than pruning, suggesting that pruning disproportionately degrades non-English performance.

Research priorities include: (1) *fairness-aware compression* that includes demographic parity metrics as part of the compression objective; (2) *representation auditing* of compressed models

across language, dialect, gender, and race; and (3) *minimum performance floors* for safety-critical applications, ensuring compressed models meet baseline accuracy thresholds for all user populations.

S14.5 Sustainable AI: SLMs as a Green AI Alternative

The energy consumption of training and serving frontier LLMs has become a significant environmental and economic concern [109, 125]. Training GPT-3 emitted an estimated 552 metric tons of CO_2 equivalent [110], and training a model two orders of magnitude smaller from scratch needs correspondingly less compute. The defensible claim about inference is conditional: *for a task a small model completes correctly, on hardware it keeps busy, serving it with the small model uses less inference energy* (D). Whether a shift to small models reduces total system energy depends on teacher-training energy, the compression pipeline, device embodied carbon, regional grid intensity, update distribution, utilization, and the rebound effect, and no published accounting answers it. Supplementary Section S16 sets out the accounting boundary.

S14.6 Democratization and Sustainability

Running a 3B model with strong reported grade-school-mathematics accuracy on an inexpensive single-board computer is a qualitative change in access. Organizations facing cloud-API cost, data-sovereignty rules, or unreliable connectivity can run capable NLP locally with no per-query charge and, subject to the scope conditions of Section 6.1, without transferring prompt content to a processor outside their control. The environmental side of the argument is conditional in the way set out above and in Supplementary Section S16: we make a per-task inference claim and no aggregate one.

S15 Open Research Directions

Beyond direct empirical verification, several promising research directions emerge from this review:

Multilingual SLM evaluation Extending the consolidated performance evidence to non-English benchmarks would reveal whether the English-centric performance hierarchy reported in the literature holds across languages, particularly for models with multilingual training data (Qwen2.5, Gemma 3, BLOOM).

On-device continual learning Developing methods for incremental adaptation of deployed SLMs to new data without full retraining would enable personalization and domain adaptation at the edge.

SLM ensembles vs. single LLM Investigating whether ensembles of specialized SLMs (one for math, one for code, one for general knowledge) with a lightweight router can match single-LLM performance at lower aggregate cost.

Multimodal SLMs Evaluating vision-language SLMs such as Phi-3-vision, Gemma 3 with vision, and Gemma 3n [48] on multimodal benchmarks would extend the comparison to the rapidly growing multimodal inference domain.

Hybrid architectures The emergence of non-transformer SLM architectures (SambaY [115], Gated Delta Networks [140]) suggests that the transformer's dominance may not extend to the edge, where linear-time inference is particularly valuable.

Federated fine-tuning Developing privacy-preserving fine-tuning protocols that aggregate gradient updates across edge devices would enable collaborative model improvement without centralized data collection.

S16 What the Environmental Claim Covers

Environmental claims for small models are routinely made at a level of generality the accounting does not support. A statement that many small-model instances on consumer hardware use less total energy than one multi-GPU server, for example, cannot be sourced or derived: its truth depends on utilization on both sides, and a server at high batch utilization is considerably more energy-efficient per token than a consumer device performing occasional single-stream inference.

A complete comparison of a small-model deployment against a large-model one has to include the training energy of the small model; the training energy of any teacher used to distill it, which can dominate and which vendors do not report; the energy of the distillation, quantization, or pruning pipeline; the embodied carbon of the devices inference runs on, amortized over their service life and the share of it spent on inference; the marginal grid intensity where each side runs, which differs by more than a factor of ten between regions; and the energy of model updates, which for on-device models means redistributing weights to every device.

Almost none of these terms is reported. The available measurements are per-query and partial: a 2023 estimate put a frontier query at about ten times the energy of a web search [29], while a 2025 production measurement covering accelerators, host systems, idle capacity, and data-centre overhead reports 0.24 Wh for the median text prompt of one large assistant [35]. Per-token compute scales with active parameters, so for a query both can answer a small model performs one to two orders of magnitude less compute than a frontier model; whether that becomes an energy saving depends on the utilization of the hardware each runs on (D). The defensible claim is therefore conditional: *for a task a small model completes correctly, on hardware it keeps busy, serving it with the small model uses less inference energy.* Whether a shift to small models reduces total system energy is a question no published accounting answers, and the rebound effect, that cheaper inference induces more of it, cuts against it.

S17 Data Provenance and Supplementary Protocols

This appendix supports the reproducibility of the *review itself*. It records where the quantitative material in Sections 5 and 6 comes from, and documents the evaluation and compression conventions the reviewed sources report, so that a reader can judge any individual figure against the conditions behind it. Device specifications are in Table S6. The appendix reports no original experimental runs.

S17.1 Provenance of the Consolidated Figures

Provenance is recorded at two levels. *Model-level* provenance fixes, for each of the 18 models in the cohort, the primary release document from which its architectural specification and the vendor-reported scores quoted in Section 5.2 derive (Table S10). *Source-class* provenance records, for each table retained in the paper, the category of source its contents are drawn from (Section S17.2). Benchmark definitions and metric variants follow the originating task papers cited in Section S9.1.

S17.2 Provenance Status by Table

Every quantitative table retained in this paper falls into one of three categories, and the category determines how much weight it can carry.

Sourced from primary release documents. The architectural specifications in Table S4 are taken from the release document listed for each model in Table S10. Layer counts, hidden sizes, attention configurations, context lengths, vocabulary sizes and training-token counts are stated directly in those documents and are stable across printings.

Table S10. Model-level provenance for the reviewed cohort: the primary release document from which each model's architectural specification and headline reported scores derive.

Model(s)	Developer	Primary source	Source type
TinyLlama-1.1B	StatNLP / SUTD	Zhang et al. [143]	Technical report
SmolLM2-1.7B	Hugging Face	Allal et al. [6]	Technical report
StableLM 2-1.6B	Stability AI	Stability AI [124]	Technical report
Qwen2.5-0.5B/1.5B/3B/7B	Alibaba	Qwen Team, Alibaba Group [112]	Technical report
Llama 3.2-1B/3B	Meta	Llama Team, AI @ Meta [82]	Technical report
Gemma 2-2B/9B	Google DeepMind	Gemma Team, Google DeepMind [43]	Technical report
Gemma 3-1B/4B	Google DeepMind	Gemma Team, Google DeepMind [45]	Technical report
Mistral 7B	Mistral AI	Jiang et al. [62]	Technical report
Phi-3-mini	Microsoft	Abdin et al. [2]	Technical report
Phi-3.5-mini	Microsoft	Microsoft [94]	Technical report
Phi-4-mini	Microsoft	Microsoft et al. [96]	Technical report
Phi-4	Microsoft	Abdin et al. [3]	Technical report

Computed from stated quantities. The weight footprints in Table S3 follow arithmetically from parameter count and an effective number of bits per weight (two bytes per parameter for FP16, 4.4–5.8 effective bits for the four-bit schemes, as stated in its caption) and are estimates, not measurements. The device specifications in Table S5 and Table S6 come from manufacturer datasheets.

Structural properties of a technique. Table 2 states what each compression technique reduces by construction. Its entries are consequences of how the technique works, not measurements: the memory factor of n-bit quantization follows from the bit-width, and the parameter factor of s-fraction pruning follows from the sparsity level. Where a cell reports a figure from a specific study, that study is cited in the cell.

Inventory-derived. The counts in the text, Table S1, the inventory appendix, and every generated figure are computed from version 3.6 of the released inventory by the released scripts.

Transcribed coverage. The coverage ledger (Table S2) records whether, and under which stated protocol, each model's primary release document or official model card reports each benchmark; the document and table for every cell are listed in `audit_coverage_ledger.csv`. For the Phi models these are the official model cards [95], and for Llama 3.2 the official model card [90].

Qualitative synthesis of the reviewed literature. The quality-cost statements in Table 3, the framework comparison in Table S8, the survey comparison in Table 1, the taxonomy summary in Table S7, and the agent-evidence summary in Table 5 summarize the works cited in their respective sections. They are positional rather than metric, and each row of the two graded tables carries the class of evidence standing behind it (Section 5.1.2).

Not reported by design. No table in this paper gives a single compression ratio or accuracy-retention percentage per technique, for the reasons in Section 4.1, and no table reports a cross-model accuracy comparison, because, as Section 5.2 and Table S2 show, such a table cannot be assembled from vendor reporting without mixing incompatible protocols and separately reported checkpoints. A partial table from single-harness third-party evaluations is the defensible alternative.

S17.3 Prompt Formats in the Source Literature

This subsection describes the conventions the reviewed sources use; it does not describe a protocol applied in this review, which ran no evaluations. The sources commonly preserve each model's own instruction wrapper while following a broadly similar task-payload structure. **Multiple-choice question answering** presents the question followed by the enumerated options (A–D) and an "Answer:" cue, with the model expected to emit a single option letter. **Mathematical reasoning**

prepends an instruction to solve the problem step by step and appends a "Let's think step by step" cue, so that a chain-of-thought precedes the final numeric answer. **Extractive question answering** supplies the context passage, then the question, then an instruction to answer from the context and to respond "unanswerable" when the context does not contain the answer. **Abstractive summarization** supplies the source article preceded by an instruction to produce a two- to three-sentence summary. Where a source reports few-shot results, it typically prepends a fixed set of exemplars of its own choosing, which differs from source to source. Because sources differ in the exemplars, in the exact wording of these wrappers, and in answer-extraction heuristics, figures from different sources are not comparable, which is part of the finding of Section 5.2.

S17.4 Quantization and Compression Pipeline

The compression workflows reported across the reviewed sources are summarized here. The reviewed literature reports five recurring precision or compression settings:

FP16 baseline Standard half-precision inference used as the reference condition.

GGUF Q4_K_M Mixed-precision 4-bit quantization via `llama.cpp`, assigning higher precision to the most sensitive tensors.

AWQ 4-bit Activation-aware weight quantization with group size 128 and calibration on held-out text sequences.

GPTQ 4-bit Layer-wise post-training quantization using second-order curvature approximations.

bitsandbytes NF4 4-bit NormalFloat quantization with double quantization, matching the QLoRA training regime.

Some sources also report SparseGPT [38]-induced 50% unstructured sparsity applied before 4-bit quantization. We do not tabulate those figures, because support for sparse inference remains hardware dependent and the setting is not uniformly reported across the reviewed sources.

S17.5 Human-Evaluation Conventions in the Source Literature

For subjective tasks such as summarization quality and instruction following, the reviewed sources that report human judgments typically rate outputs along three axes (**relevance**, **coherence**, and **factual accuracy**) on a 1–5 Likert scale, with model identity hidden from annotators and inter-annotator reliability reported via a Krippendorff-style coefficient. MT-Bench-style results substitute a strong LLM judge for human raters. Because these protocols differ across sources in rubric wording and judge model, and because per-model coverage is sparse, instruction-following and summarization are not tabulated in this review and are discussed only qualitatively.

S18 Broader Small-Model Ecosystem Context

This appendix situates the reviewed SLM cohort within the wider model ecosystem, which matters because the 2025–2026 release cycle moved compact open models, open-weight reasoning models, and frontier API systems in parallel rather than in sequence. The summary below records the ecosystem context needed to interpret the cohort, followed by the full generated variant inventory.

S18.1 Landscape Snapshot

Three ecosystem trends are especially relevant to the present study:

Capability compression Reasoning, long-context handling, and multimodality have moved into the 1B–8B range, narrowing the gap between deployable SLMs and cloud-scale LLMs.

Open-weight expansion Families such as Qwen, Phi, Gemma, SmolLM, DeepSeek distills, Granite, and MiniCPM have expanded the design space for self-hosted deployment (Figure S9).

Deployment diversification Inference targets now span cloud GPUs, consumer GPUs, laptops, smartphones, browsers, and embedded edge boards, making model choice inseparable from hardware choice.

The small-active-parameter turn The most distinctive feature of the mid-2026 wave is how much of it activates 3–13B parameters against totals of 30B to 284B: Nemotron 3.5 Lightning (30B/3B), Hy-MT2-30B-A3B, North Mini Code (30B/3B), Laguna XS 2.1 (33B/3B), Ling 3.0 Flash (124B/5.1B) and its domain variants, and DeepSeek V4 Flash (284B/13B). These are the models that make the C1/C2 distinction load-bearing rather than pedantic, since each is cheap per token and none is resident on a phone.

The May–September 2026 additions to the inventory illustrate both ends of the range. At the small end, IBM shipped Granite 4.1 and then Granite 4.2 at 3B and 8B with a thinking switch, Liquid AI extended LFM2.5 from 230M to an 8.3B/1.5B on-device MoE, and Tencent released the Hy-MT2 translation family at 1.8B and 7B. Frontier API releases continued in parallel over the same months; they are recorded, with their sources, in the LLM part of the released inventory, and none of them bears on a claim in this paper.

Figure S7 gives a size-proportional overview of the inventory by developer and family, and Figure S8 tracks the entry of new developers into the open-SLM space.

S18.2 Representative Family Reference

Table S11 lists the representative families tracked in the inventory, with their developers, size ranges, access, and the role each plays in the landscape.

Table S11. Representative model families tracked in the inventory.

Family	Developer	Representative Versions	Access	Typical Role
Qwen2.5 / Qwen3	Alibaba	0.5B–14B dense, MoE variants	Open weights	Strong multilingual, math, and code performance
Phi-3 / Phi-4	Microsoft	3.8B mini variants, 14B reasoning	Open weights	Data-curated reasoning-focused SLMs
Gemma 2 / 3 / 3n	Google DeepMind	1B–12B dense, mobile-first multimodal variants	Open weights	Distilled compact models
Llama 3.2 / 4	Meta	1B–3B edge models, larger MoE frontier models	Open weights	Broad deployment ecosystem from edge to server
SmolLM2 / SmolLM3	Hugging Face	135M–3B	Open weights	Lightweight, fully open research and edge baselines
DeepSeek distills	DeepSeek	1.5B–14B distilled reasoning models	Open weights	Reasoning transfer into self-hosted sizes
Granite 3 / 4	IBM	2–8B dense; 4.0 hybrid Mamba-2/MoE (350M–32B, 1–9B active)	Open weights	Enterprise-friendly, tool-use and RAG focus
Falcon-H1	TII	0.5–34B parallel attention/Mamba-2 hybrid	Open weights	Long-context edge-to-server
LFM2 / LFM2.5	Liquid AI	230M–8.3B hybrid conv/attention, incl. 8B-A1B MoE	Open weights	CPU-oriented on-device models
MiniCPM	OpenBMB	Compact multimodal variants	Open weights	Edge multimodal use cases
Ling / Nemotron 3.5 / Laguna	inclusionAI / NVIDIA / Poolside	Small-active MoE, 3–13B active	Open weights	SLM-like per-token cost at LLM-like residency
Frontier API families	OpenAI / Anthropic / Google / xAI / others	API frontier models (see released inventory)	API only	High-end proprietary baselines and teacher models

S18.3 Classification Conventions

The inventory applies conditions C1 and C2 of Section 1.2.1 from the parameter counts each record carries:

- A record enters the inventory when its *active* parameter count is at or below roughly 20B (C1); dense models, for which active equals total, therefore enter at or below 20B total. C2 (total at or below 20B) then separates SLMs from small-active-parameter MoEs; both are listed, and only the former carry the paper's storage, resident-memory, and on-device claims.
- Where a source states no active-parameter count for a model it describes as a mixture of experts, the record is marked undisclosed rather than having active set equal to total. Nested legacy records are read from their active field.

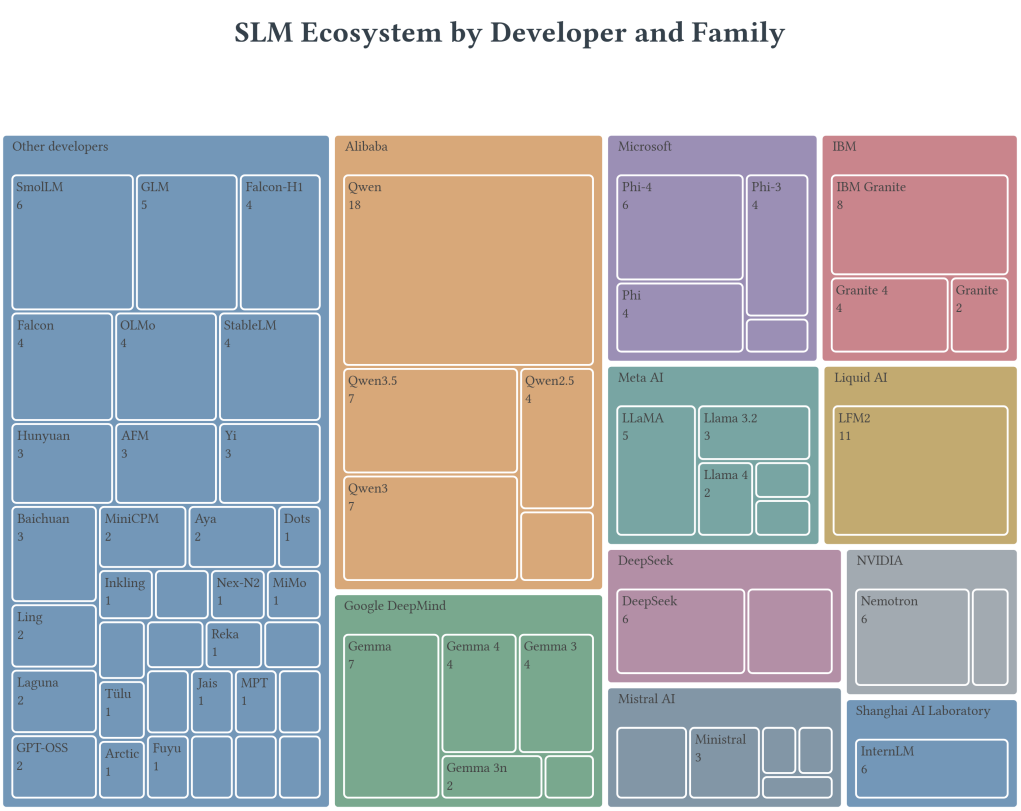


Fig. S7. Documented SLM variants grouped by developer and family; area is proportional to the number of variants. Generated from `merged_llm_database.json` (version 3.6; releases through 4 September 2026).

- Per-layer-embedding "effective" sizes (Gemma 3n, Gemma 4 E2B/E4B) are not sparse activation; C1 is evaluated on the total and the effective size is kept as a note.
- M1 is satisfied when a developer source states size and architecture; the source check of Supplementary Section S2.2 records which source that is. Records of undisclosed size, whatever their marketing, are kept in the comparison part of the file as `undisclosed`.
- Encoder-decoder models, derivative fine-tunes without a technical report, and API aliases are excluded by named editorial decision and kept under `models.excluded` with their reason.

The count history from 126 to the present 209 is given in Supplementary Section S2.3, and the released changelog names every record added, moved, or removed at each step.

S18.4 Generated Variant Inventory

The following inventory is generated directly from `merged_llm_database.json` using `scripts/generate_model_tables.py`. Version 3.6 of the database covers 209 inventory records (179 SLMs and 30 small-active-parameter MoEs), 223 comparison records (104 of disclosed size that fail C1 and 119 of undisclosed size), and 7 excluded candidates, 439 records in total, for releases through 4 September 2026. Only the inventory is reproduced here (Table S12), organized by release year and family, with

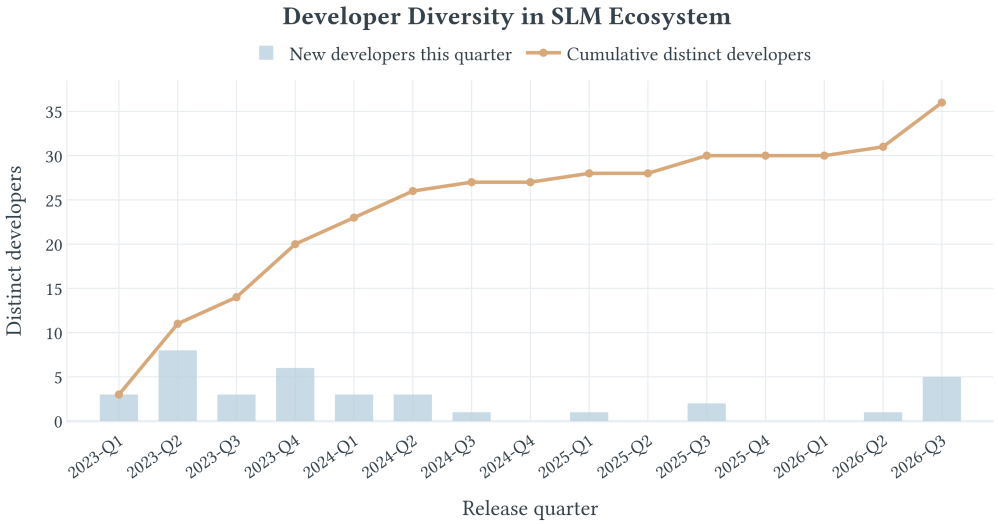


Fig. S8. Cumulative distinct developers releasing documented SLM variants (line) and new developers per quarter (bars).

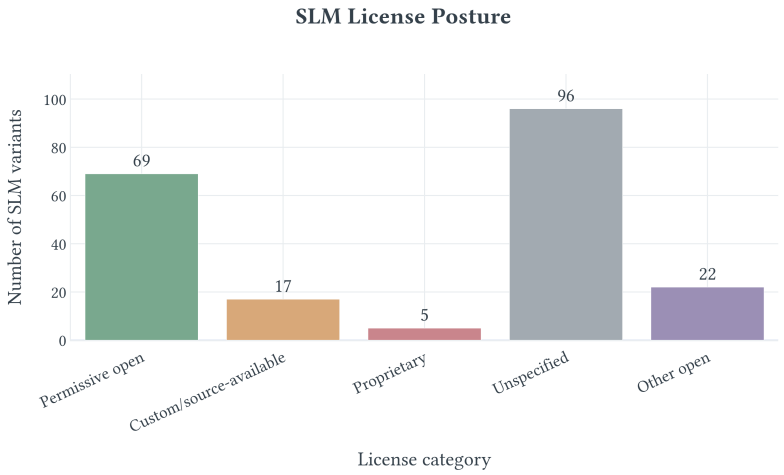


Fig. S9. License posture of documented SLM variants. Permissive open licenses (Apache-2.0, MIT) account for the plurality; "unspecified" entries are typically research checkpoints without an attached license file.

total and, where different, active parameters given in the size column; the comparison records are in the released file.

Each record carries the fields `meets_c1_active_le_20b`, `meets_c2_total_le_20b`, `slm_class`, `inclusion_decision`, `developer_source_found`, and `source_check`, so that a reader can recompute the inventory under a different threshold, restrict it to models satisfying both conditions, or audit each record's source.

Chronology. Inventory: 2026, 2025, 2024, 2023 and earlier.

Table S12. Generated chronological small language model inventory from `merged_llm_database.json`, grouped by release year and family. Entries whose parameter column reads "NB (MB act.)" with $N > 20$ are small-active-parameter MoEs (C1 only), not SLMs, and are excluded from every storage and on-device claim.

Model	Access	Params	Context	Modalities	License	Release	Notes
2026							
Qwen3.5 — Alibaba (Qwen3.5 (Mar 2026)) [family page]							
Qwen3.5-0.8B	OW	0.8B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	ultra-lightweight multimodal; on-device
Qwen3.5-2B	OW	2B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	on-device multimodal chat; edge AI
Qwen3.5-4B	OW	4B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	mobile multimodal assistant; agentic coding
Qwen3.5-9B	OW	9B	256K	Text+Vision+Video	Apache 2.0	2026-03-02	strong multimodal SLM; coding
Qwen3.5-35B-A3B	OW	35B (3B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	efficient MoE for edge/server; multimodal reasoning
Qwen3.5-122B-A10B	OW	122B (10B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	strong MoE reasoning; locally runnable flagship
Qwen3.5-397B-A17B	Both	397B (17B act.)	256K	Text+Vision	Apache 2.0	2026-02-16	flagship open-weight multimodal; 201 languages
AFM — Apple (AFM) [family page]							
Apple AFM 3 Core Advanced	API	20B (1-4B act.)	64K	Text+Vision	Proprietary	2026-06-08	on-device assistant; multimodal understanding
Command — Cohere (Command) [family page]							
North Mini Code 1.0	OW	30B (3B act.)	250K	Text	CC-BY-NC (Cohere Labs)	2026-06-17	coding; agentic coding
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek V4 Flash Vision Exp	OW	284B (13B act.)	1M	Text+Vision	See source	2026-08-21	vision; long context
DeepSeek V4 Flash 0731	OW	284B (13B act.)	1280K	Text	See source	2026-07-31	general; long context
Dots — Dots Studio (Dots3-Note Preview (Aug 2026)) [family page]							
Dots3-Note Preview	OW	280B (16B act.)	500K	Text	Apache 2.0	2026-08-14	note taking; general
Gemma 4 — Google DeepMind							
Gemma 4 26B (MoE)	OW	26B (3.8B act.)	128K	Text+Vision	Apache 2.0	2026-04-02	reasoning; multimodal
Gemma 4 E2B	OW	~5B	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	on-device; multimodal
Gemma 4 E4B	OW	~8B	128K	Text+Vision+Audio	Apache 2.0	2026-04-02	multimodal; reasoning
Gemma 4 12B	OW	12B	256K	Text+Vision+Audio	Apache 2.0	2026-04-02	multimodal; reasoning
Granite 4 — IBM							
Granite 4.2 3B	OW	3B	128K	Text	Apache 2.0	2026-08-25	reasoning; on-device
Granite 4.2 8B	OW	8B	128K	Text	Apache 2.0	2026-08-25	reasoning; agentic
Granite 4.1 3B	OW	3B	128K	Text	Apache 2.0	2026-04-29	enterprise; on-device
Granite 4.1 8B	OW	8B	128K	Text	Apache 2.0	2026-04-29	enterprise; agentic

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
LFM2 — Liquid AI							
LFM2.5-2.6B	Both	2.69B	128K	Text	LFM1.0	2026-08-04	agentic workflows; data extraction
LFM2.5-8B-A1B	OW	8.3B (1.5B act.)	128K	Text	LFM1.0	2026-05-28	on-device; tool calling
LFM2.5-350M	OW	350M	128K	Text	LFM1.0	2026	on-device; edge
LFM2.5-1.2B	OW	1.2B	128K	Text	LFM1.0	2026	on-device; edge
LFM2.5-Audio-1.5B	OW	1.5B	–	Text+Audio	LFM1.0	2026	on-device speech; transcription
LFM2.5-VL-3B	OW	3B	–	Text+Vision	LFM1.0	2026	on-device VLM; document extraction
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral Small 4 (119B)	Both	119B (6.5B act.)	256K	Text+Vision	Apache 2.0	2026-03-16	hybrid reasoning; coding
NVIDIA Nemotron — NVIDIA (Nemotron 3 Super (Mar 2026)) [family page]							
NemotronLabs VoiceChat 11B	OW	11B	128K	Text+Audio	NVIDIA Custom	2026-08-03	voice interaction; on-device speech
Nemotron 3 Super	OW	120B (12B act.)	1M	Text	NVIDIA Open Model License	2026-03	open hybrid Mamba-Transformer MoE; agentic reasoning
Nemotron — NVIDIA (Nemotron) [family page]							
Nemotron 3.5 Lightning 30B-A3B	OW	30B (3B act.)	256K	Text	NVIDIA Open Model License	2026-08-11	agentic execution layer; high-volume tool calling
Nemotron 3.5 Content Safety	OW	4B	128K	Text+Vision	NVIDIA Open Model License	2026-06-04	guardrails; content moderation
Nex-N2 — Nex AGI (Nex-N2-Mini (Jun 2026)) [family page]							
Nex-N2-Pro	OW	397B (17B act.)	256K	Text+Vision	See source	2026-06-08	agentic; coding
Laguna — Poolside (Laguna S 2.1 (Jul 2026)) [family page]							
Laguna S 2.1	OW	118B (8B act.)	1M	Text	See source	2026-07-21	coding agent
Laguna XS 2.1	OW	33B (3B act.)	256K	Text	See source	2026-07-02	coding agent
Hunyuan — Tencent (Hunyuan) [family page]							
Hy-MT2-1.8B	OW	1.8B	8K	Text	Tencent Hunyuan License	2026-08-20	translation; on-device
Hy-MT2-30B-A3B	OW	30B (3B act.)	8K	Text	Tencent Hunyuan License	2026-08-20	translation
Hy-MT2-7B	OW	7B	8K	Text	Tencent Hunyuan License	2026-08-19	translation; Lang: 33 language pairs + 5 dialect/minority pairs
Inkling — Thinking Machines Lab (Inkling Small (Jul 2026)) [family page]							
Inkling Small	OW	276B (12B act.)	1M	Text+Vision	See source	2026-07-30	general; multimodal
MiMo — Xiaomi (MiMo-V2-Pro (Mar 2026)) [family page]							
MiMo-V2-Flash	OW	309B (15B act.)	256K	Text	Undisclosed	2026	high-throughput coding agents; production serving
Ling — inclusionAI (Ling-1T (2026)) [family page]							
Ling 3.0 Flash Fin	OW	124B (5.1B act.)	256K	Text	MIT	2026-08-27	finance
Ling 3.0 Flash	OW	124B (5.1B act.)	256K	Text	See source	2026-07-23	general; agentic

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
2025							
Qwen — Alibaba (Qwen) [family page]							
Qwen2.5-Omni-7B	OW	7B	–	Text+Audio+Video+Image	See source	2025-03-26	omni-modal; text and audio output
Qwen2.5-VL-7B	OW	7B	–	Text+Video+Image	See source	2025-01-22	active
Qwen3 — Alibaba (Qwen3 (Apr 2025)) [family page]							
Qwen3-4B-Instruct-2507	OW	4B	128K	Text	Apache 2.0	2025-07	improved instruction following; non-thinking mode optimised
Qwen3-0.6B	OW	0.6B	32K	Text	Apache 2.0	2025-04-29	edge inference; IoT
Qwen3-1.7B	OW	1.7B	32K	Text	Apache 2.0	2025-04-29	mobile inference; Q&A
Qwen3-30B-A3B	OW	30B (3B act.)	128K	Text	Apache 2.0	2025-04-29	MoE efficiency; broad capability in tiny footprint
Qwen3-4B-Instruct	OW	4B	128K	Text	Apache 2.0	2025-04-29	on-device chat; coding
Qwen3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04-29	single-GPU inference; reasoning
Qwen3-14B-Instruct	OW	14B	128K	Text	Apache 2.0	2025-04-29	high-quality on-premise chat; coding
AFM — Apple (AFM) [family page]							
Apple Foundation Model (On-Device, 2nd gen)	API	~3B	64K	Text+Vision	Proprietary	2025-06-09	on-device assistant; writing tools
DeepSeek R1-Distill — DeepSeek (R1-Distill (Jan 2025)) [family page]							
DeepSeek-R1-Distill-Qwen-1.5B	OW	1.5B	128K	Text	MIT	2025-01-20	reasoning at ultra-small scale; math
DeepSeek-R1-Distill-Qwen-7B	OW	7B	128K	Text	MIT	2025-01-20	chain-of-thought reasoning; coding
DeepSeek-R1-Distill-Llama-8B	OW	8B	128K	Text	MIT	2025-01-20	reasoning on Llama 3 base; on-premise
DeepSeek-R1-Distill-Qwen-14B	OW	14B	128K	Text	MIT	2025-01-20	strong reasoning 14B; top distill for coding
Gemma 3 — Google DeepMind (Gemma 3 (Mar 2025)) [family page]							
Gemma 3 270M-IT	OW	270M	32K	Text	Google Gemma License	2025-03	ultra-lightweight prototyping; experiments
Gemma 3 1B-IT	OW	1B	32K	Text	Google Gemma License	2025-03	on-device mobile inference; lightweight Q&A
Gemma 3 4B-IT	OW	4B	128K	Text+Vision	Google Gemma License	2025-03	multimodal on-device; image understanding
Gemma 3 12B-IT	OW	12B	128K	Text+Vision	Google Gemma License	2025-03	high-quality assistant; 140+ languages
Gemma 3n — Google DeepMind (Gemma-3n (2025)) [family page]							
Gemma-3n-E2B-IT	OW	~5B	32K	Text+Vision+Audio+Video	Google Gemma License	2025	mobile-first multimodal; on-device speech/vision/text
Gemma-3n-E4B-IT	OW	~8B	32K	Text+Vision+Audio+Video	Google Gemma License	2025	higher-accuracy mobile multimodal; edge devices
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM3-3B	OW	3B	64K	Text	Apache 2.0	2025	dual-mode reasoning; tool calling

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Granite — IBM (Granite) [family page]							
Granite 3.2-2B	OW	2B	125K	Text	See source	2025-02-26	Thinking mode; 128K context
Granite 3.2-8B	OW	8B	125K	Text	See source	2025-02-26	Thinking mode (chain-of-thought); 128K context
IBM Granite — IBM (Granite 3.3 (Apr 2025)) [family page]							
Granite 4.0 Nano 350M	OW	350M	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 Nano 1B	OW	1B	128K	Text	Apache 2.0	2025-10-28	on-device; browser
Granite 4.0 H-Tiny	OW	7B (1B act.)	128K	Text	Apache 2.0	2025-10-02	low-latency; agentic
Granite 4.0 H-Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; tool use
Granite 4.0 Micro	OW	3B	128K	Text	Apache 2.0	2025-10-02	edge; RAG
Granite 4.0 H-Small	OW	32B (9B act.)	128K	Text	Apache 2.0	2025-10-02	enterprise; agentic
Granite 3.3-2B-Instruct	OW	2B	128K	Text	Apache 2.0	2025-04	enterprise NLP; governed AI
Granite 3.3-8B-Instruct	OW	8B	128K	Text	Apache 2.0	2025-04	enterprise reasoning; Fill-in-Middle code
LFM2 — Liquid AI							
LFM2-8B-A1.5B (MoE)	OW	8.3B (1.5B act.)	32K	Text	LFM Open License v1.0	2025-08	on-device; agentic
LFM2-2.6B	OW	2.6B	32K	Text	LFM Open License v1.0	2025-08	on-device
LFM2-350M	OW	350M	32K	Text	LFM Open License v1.0	2025-07-10	on-device; low latency
LFM2-700M	OW	700M	32K	Text	LFM Open License v1.0	2025-07-10	on-device
LFM2-1.2B	OW	1.2B	32K	Text	LFM Open License v1.0	2025-07-10	on-device; agentic
Llama 4 — Meta AI (Llama 4 Maverick (Apr 2025)) [family page]							
Llama 4 Maverick	Both	400B (17B act.)	10M	Text+Vision	Llama 4 Community License	2025-04-05	image understanding; coding
Llama 4 Scout	Both	109B (17B act.)	10M	Text+Vision	Llama 4 Community License	2025-04-05	extreme long-context; single H100 runnable
BitNet — Microsoft (BitNet) [family page]							
BitNet b1.58 2B4T	OW	2B	–	Text	MIT	2025-04-16	1-bit LLM (ternary weights: -1, 0, 1); Extreme efficiency
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4-mini-flash-reasoning	OW	3.8B	128K	Text	MIT	2025-04	fast reasoning; edge agents
Phi-4-mini-reasoning	OW	3.8B	128K	Text	MIT	2025-04	chain-of-thought reasoning; math
Phi-4-reasoning	OW	14B	128K	Text	MIT	2025-04	complex reasoning; STEM
Phi-4-reasoning-plus	OW	14B	128K	Text	MIT	2025-04	RL-trained reasoning; math olympiad-level
Phi-4-mini-instruct	OW	3.8B	128K	Text	MIT	2025-02	on-device chat; function calling
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral 3 (14B)	OW	14B	–	Text+Vision	See source	2025-12-02	edge deployment; multimodal
Ministral-3B-Instruct-2512	OW	3.8B	256K	Text+Vision	Apache 2.0	2025-12	tiny multimodal; function calling

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Voxtral — Mistral AI (Voxtral) [family page]							
Voxtral Mini 3B	OW	4.7B	–	Text+Audio	Apache 2.0	2025-07-15	speech understanding; transcription
Nemotron — NVIDIA (Nemotron) [family page]							
Nemotron 3 Nano-30B	OW	30B	–	Text	See source	2025-12-15	hybrid Mamba-Transformer MoE; 1M context
NVIDIA Nemotron Nano 2	OW	9B	128K	Text	See source	2025-08-18	on-device; efficient
GPT-OSS — OpenAI (gpt-oss-120b (2025)) [family page]							
gpt-oss-20b	OW	21B (3.6B act.)	128K	Text	Apache 2.0	2025	edge reasoning; 16GB device inference
gpt-oss-120b	OW	117B (5.1B act.)	128K	Text	Apache 2.0	2025	open-weight reasoning; near o4-mini parity
MiniCPM — OpenBMB (MiniCPM-V 4.5 (2025)) [family page]							
MiniCPM-V 4.5	OW	~8B	32K	Text+Vision	Apache 2.0	2025	mobile vision-language; on-device VQA
MiniCPM-o 3.0	OW	~9B	32K	Text+Vision+Audio	Apache 2.0	2025	end-to-end multimodal on-device; live streaming
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM3-8B	OW	8B	32K	Text	See source	2025-01-09	Reasoning via deep thinking; Knowledge-rich training pipeline
Falcon-H1 — Technology Innovation Institute							
Falcon-H1-0.5B	OW	0.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-1.5B	OW	1.5B	256K	Text	Falcon-LLM License 2.0	2025-07-31	edge; long context
Falcon-H1-3B	OW	3B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
Falcon-H1-7B	OW	7B	256K	Text	Falcon-LLM License 2.0	2025-07-31	general; long context
2024							
Yi — 01.AI (Yi) [family page]							
Yi-1.5-6B	OW	6B	–	Text	See source	2024-05-13	legacy
Yi-1.5-9B	OW	9B	–	Text	See source	2024-05-13	legacy
Qwen — Alibaba (Qwen) [family page]							
Qwen2.5-Coder-1.5B	OW	1.5B	–	Text	See source	2024-11-12	compact coding model
Qwen2.5-14B	OW	14B	–	Text	See source	2024-09-19	active
Qwen2-VL-7B	OW	7B	–	Text+Video+Image	See source	2024-08-28	vision-language; video understanding
Qwen2-Audio-7B	OW	7B	–	Text+Audio	See source	2024-08-09	audio understanding; speech recognition
Qwen2-0.5B	OW	0.5B	–	Text	See source	2024-06-07	active
Qwen2-1.5B	OW	1.5B	–	Text	See source	2024-06-07	active
Qwen2-7B	OW	7B	–	Text	See source	2024-06-07	active
Qwen2-57B-A14B	OW	57B	–	Text	See source	2024-06-07	active
Qwen1.5-MoE-A2.7B	OW	14.3B	–	Text	See source	2024-03-28	MoE architecture; efficient

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Qwen1.5-0.5B	OW	0.5B	–	Text	See source	2024-02-05	legacy
Qwen1.5-7B	OW	7B	–	Text	See source	2024-02-05	legacy
Qwen1.5-14B	OW	14B	–	Text	See source	2024-02-05	legacy
Qwen2.5 — Alibaba (Qwen2.5 (Sep 2024)) [family page]							
Qwen2.5-0.5B-Instruct	OW	0.5B	32K	Text	Apache 2.0	2024-09-19	extreme edge/IoT; classification
Qwen2.5-1.5B-Instruct	OW	1.5B	32K	Text	Apache 2.0	2024-09-19	edge NLP; lightweight assistants
Qwen2.5-3B-Instruct	OW	3B	32K	Text	Apache 2.0	2024-09-19	mobile chat; summarization
Qwen2.5-7B-Instruct	OW	7B	128K	Text	Apache 2.0	2024-09-19	general assistant; coding
Qwen2.5-Coder — Alibaba (Qwen2.5-Coder (Nov 2024)) [family page]							
Qwen2.5-Coder-7B-Instruct	OW	7B	128K	Text+Code	Apache 2.0	2024-11	code generation; debugging
Qwen2.5-Coder-14B-Instruct	OW	14B	128K	Text+Code	Apache 2.0	2024-11	advanced code assistant; agentic code tasks
OLMo — Allen Institute for AI (Ai2) (OLMo) [family page]							
OLMo 2-7B	OW	7B	–	Text	See source	2024-11-26	fully open; training code included
OLMo 2-13B	OW	13B	–	Text	See source	2024-11-26	active
OLMo-1B	OW	1B	2K	Text	See source	2024-02-01	Fully open (data, code, weights, logs); Dolma training data
OLMo-7B	OW	7B	–	Text	See source	2024-02-01	fully open; training data/code/logs released
Tülu — Allen Institute for AI (Ai2) (Tülu) [family page]							
Tülu 3-8B	OW	8B	–	Text	See source	2024-11-22	instruction following; DPO
AFM — Apple (AFM) [family page]							
Apple Foundation Model (On-Device)	API	~3B	4K	Text	Proprietary	2024-06-10	On-device Apple Intelligence; Privacy-preserving
OpenELM — Apple (OpenELM) [family page]							
OpenELM	OW	3B	–	Text	See source	2024-04-24	layer-wise scaling; on-device
Aya — Cohere (Aya) [family page]							
Aya Expanse-8B	OW	8B	8K	Text	See source	2024-10-22	Improved multilingual performance; Data arbitrage and synthetic data strategies
Aya 23-8B	OW	8B	8K	Text	See source	2024-05-23	23 language support; Multilingual instruction following
Cohere Command — Cohere (Command A (Mar 2025)) [family page]							
Command R7B	Both	7B	128K	Text	Apache 2.0	2024-12	RAG; multilingual enterprise tasks
TinyLlama — Community (TinyLlama-1.1B (2024)) [family page]							
TinyLlama-1.1B-Chat	OW	1.1B	2K	Text	Apache 2.0	2024	ultra-light on-device chat; prototyping
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-VL-7B	OW	7B	4K	Text+Vision	See source	2024-03-13	Vision-language model; Hybrid vision encoder

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
DeepSeek-Math-7B	OW	7B	4K	Text	See source	2024-02-05	Math-specialized; Group Relative Policy Optimization (GRPO)
Gemma — Google DeepMind (Gemma) [family page]							
Gemma 2-2B	OW	2B	8K	Text	See source	2024-07-31	Knowledge distillation from larger model; Sliding window attention
Gemma 2-9B	OW	9B	–	Text	See source	2024-06-27	improved efficiency; knowledge distillation
PaliGemma	OW	3B	8K	Text+Vision	See source	2024-05-14	Vision-language model; SigLIP vision encoder + Gemma 2B
RecurrentGemma-2B	OW	2B	8K	Text	See source	2024-04-09	Griffin architecture with linear recurrence; Efficient inference
CodeGemma-7B	OW	7B	–	Text	See source	2024-04-09	code generation; code completion
Gemma-2B	OW	2B	–	Text	See source	2024-02-21	legacy
Gemma-7B	OW	7B	–	Text	See source	2024-02-21	built from Gemini technology; efficient
SmolLM — Hugging Face (SmolLM3-3B (2025)) [family page]							
SmolLM2-135M	OW	135M	2K	Text	Apache 2.0	2024-10-31	ultra-micro inference; browser/WASM
SmolLM2-360M-Instruct	OW	360M	8K	Text	Apache 2.0	2024-10-31	on-device/browser inference
SmolLM2-1.7B-Instruct	OW	1.7B	8K	Text	Apache 2.0	2024-10-31	efficient on-device chat; coding assist
SmolLM-135M	OW	135M	2K	Text	See source	2024-07-17	Ultra-small LM; On-device capable
SmolLM	OW	1.7B	–	Text	See source	2024-07-16	tiny but capable; on-device
LLaMA — Meta AI (LLaMA) [family page]							
Llama-3-8B	OW	8B	–	Text	See source	2024-04-18	improved tokenizer; grouped query attention
Llama 3.1 — Meta AI (Llama 3.1 (Jul 2024)) [family page]							
Llama 3.1-8B-Instruct	OW	8B	128K	Text	Llama 3.1 Community License	2024-07-23	most fine-tuned SLM base globally; RAG
Llama 3.2 — Meta AI (Llama 3.2 (Sep 2024)) [family page]							
Llama 3.2-1B-Instruct	OW	1B	128K	Text	Llama 3.2 Community License	2024-09-25	ultra-small on-device; Android/iOS apps
Llama 3.2-3B-Instruct	OW	3B	128K	Text	Llama 3.2 Community License	2024-09-25	mobile AI; default on-device Qualcomm
Llama 3.2-11B-Vision-Instruct	OW	11B	128K	Text+Vision	Llama 3.2 Community License	2024-09-25	vision-language tasks; document understanding
Phi — Microsoft (Phi) [family page]							
Phi-3.5-MoE	OW	41.9B	–	Text	See source	2024-08-20	MoE architecture; 16 experts
Phi-3 — Microsoft (Phi-3.5 (Aug 2024)) [family page]							
Phi-3.5-mini-instruct	OW	3.8B	128K	Text	MIT	2024-08	multilingual instruction following
Phi-3-mini-128k-instruct	OW	3.8B	128K	Text	MIT	2024-04-23	on-device chat; instruction following
Phi-3-small-128k-instruct	OW	7B	128K	Text	MIT	2024-04-23	balanced reasoning & coding

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
Phi-3-medium-128k-instruct	OW	14B	128K	Text	MIT	2024-04-23	on-premise deployment; complex reasoning
Phi-4 — Microsoft (Phi-4-reasoning-plus (Apr 2025)) [family page]							
Phi-4	OW	14B	16K	Text	MIT	2024-12-12	STEM reasoning; math
Ministral — Mistral AI (Ministral-3B (Dec 2025)) [family page]							
Ministral-8B-Instruct-2410	OW	8B	128K	Text	Apache 2.0	2024-10	edge-friendly assistant; function calling
Mistral Small — Mistral AI (Mistral Small 4 (Mar 2026)) [family page]							
Mistral NeMo 12B	OW	12B	128K	Text	Apache 2.0	2024-07	enterprise RAG; long-document summarization
Mistral 7B v0.3	OW	7.3B	32K	Text	Apache 2.0	2024-05-22	general chat; RAG
Pixtral — Mistral AI (Pixtral) [family page]							
Pixtral-12B	OW	12B	–	Text+Image	See source	2024-09-17	vision-language; variable image resolution
Nemotron — NVIDIA (Nemotron) [family page]							
Minitron-8B	OW	8B	8K	Text	See source	2024-07-17	Width and depth pruning; Knowledge distillation
Nemotron-4-15B	OW	15B	8K	Text	See source	2024-03-14	High-quality training data; Strong downstream task performance
Reka — Reka AI (Reka) [family page]							
Reka Edge	API	7B	–	Text+Vision	Proprietary	2024-02-12	compact; multimodal
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM2.5-7B	OW	7B	–	Text	See source	2024-07-03	1M context; tool use
InternLM2.5-20B	OW	20B	1M	Text	See source	2024-07-03	1M context with LMDeploy; Strong reasoning
InternLM2-20B	OW	20B	–	Text	See source	2024-01-17	200K context; tool use
Arctic — Snowflake (Arctic) [family page]							
Snowflake Arctic	OW	480B	–	Text	See source	2024-04-24	enterprise-grade; 128 experts
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM 2 12B	OW	12B	4K	Text	See source	2024-02-20	Multilingual (7 languages); Strong code capability
StableLM 2 1.6B	OW	1.6B	4K	Text	Stability AI License	2024-01-19	lightweight creative writing; basic chat
Falcon — Technology Innovation Institute (TII) (Falcon 180B (Sep 2023)) [family page]							
Falcon 3	OW	10B	–	Text+Audio+Video+Image	See source	2024-12-12	multimodal; runs on laptops
Falcon Mamba-7B	OW	7B	–	Text	See source	2024-08-12	SSM architecture; constant memory usage
Falcon 2-11B	OW	11B	–	Text	See source	2024-05-14	multimodal variant; vision-to-language
GLM — Zhipu AI (GLM-5 (Feb 2026)) [family page]							
GLM-4-9B	OW	9B	–	Text	See source	2024-06-06	128K-1M context; tool use

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
CogVLM2-19B	OW	19B	8K	Text+Video+Image	See source	2024-05-24	Visual understanding; Video understanding
2023 and earlier							
Yi — 01.AI (Yi) [family page]							
Yi-6B	OW	6B	–	Text	See source	2023-11-02	bilingual; strong reasoning
Fuyu — Adept AI (Fuyu) [family page]							
Fuyu-8B	OW	8B	–	Text+Vision	See source	2023-10-17	Simplified multimodal architecture; No separate vision encoder
Qwen — Alibaba (Qwen) [family page]							
Qwen-Audio	OW	7B	–	Text+Audio	See source	2023-11-30	Audio understanding; Multi-task audio processing
Qwen-14B	OW	14B	–	Text	See source	2023-09-25	legacy
Qwen-VL	OW	9.6B	–	Text+Image	See source	2023-08-24	vision-language; image understanding
Qwen-7B	OW	7B	–	Text	See source	2023-08-03	bilingual; tool use
Baichuan — Baichuan Inc. (Baichuan) [family page]							
Baichuan-2-13B	OW	13B	–	Text	See source	2023-09-06	legacy
Baichuan-13B	OW	13B	–	Text	See source	2023-07-11	legacy
Baichuan-7B	OW	7B	–	Text	See source	2023-06-15	bilingual; strong Chinese performance
Cerebras-GPT — Cerebras Systems (Cerebras-GPT) [family page]							
Cerebras-GPT-13B	OW	13B	2K	Text	See source	2023-03-28	Chinchilla compute-optimal training; Trained on Cerebras CS-2 hardware
DeepSeek — DeepSeek (DeepSeek) [family page]							
DeepSeek-Coder-1.3B	OW	1.3B	–	Text	See source	2023-11-02	compact code model; fill-in-the-middle
DeepSeek-Coder-6.7B	OW	6.7B	–	Text	See source	2023-11-02	code generation; fill-in-the-middle
Pythia — EleutherAI (Pythia) [family page]							
Pythia-12B	OW	12B	2K	Text	See source	2023-04-03	154 intermediate checkpoints; Reproducible training
Gemini — Google DeepMind (Gemini) [family page]							
Gemini 1.0 Nano	API	3.25B	–	Text	Proprietary	2023-12-06	on-device; efficient
StarCoder — Hugging Face / BigCode (StarCoder) [family page]							
StarCoder	OW	15.5B	–	Text	See source	2023-05-04	code generation; 80+ programming languages
Jais — Inception / Core42 / Cerebras (Jais) [family page]							
Jais-13B	OW	13B	2K	Text	See source	2023-08-30	Best Arabic LLM at launch; Bilingual Arabic-English
Code Llama — Meta AI (Code Llama) [family page]							
Code Llama-7B	OW	7B	–	Text	See source	2023-08-24	code generation; infilling

continued on next page

Table S12. Generated chronological small language model inventory continued.

Model	Access	Params	Context	Modalities	License	Release	Notes
LLaMA — Meta AI (LLaMA) [family page]							
Llama-2-7B	OW	7B	–	Text	See source	2023-07-18	open weights; commercial use allowed
Llama-2-13B	OW	13B	–	Text	See source	2023-07-18	legacy
LLaMA-7B	OW	7B	–	Text	Research only	2023-02-24	efficient training; strong few-shot performance
LLaMA-13B	OW	13B	–	Text	Research only	2023-02-24	legacy
Phi — Microsoft (Phi) [family page]							
Phi-2	OW	2.7B	–	Text	See source	2023-12-13	language comprehension; reasoning
Phi-1.5	OW	1.3B	–	Text	See source	2023-09-11	reasoning; language understanding
Phi-1	OW	1.3B	–	Text	See source	2023-06-20	Python coding; textbook-quality data
Mixtral — Mistral AI (Mixtral 8x22B (Apr 2024)) [family page]							
Mixtral-8x7B	OW	46.7B	–	Text	See source	2023-12-11	MoE architecture; 8 experts
MPT — MosaicML (Databricks) (MPT) [family page]							
MPT-7B	OW	7B	–	Text	See source	2023-05-05	ALiBi positional encoding; long context
InternLM — Shanghai AI Laboratory (InternLM) [family page]							
InternLM-20B	OW	20B	–	Text	See source	2023-09-20	legacy
InternLM-7B	OW	7B	–	Text	See source	2023-07-06	bilingual; long context
StableLM — Stability AI (StableLM 2 (Jan 2024)) [family page]							
StableLM-Zephyr-3B	OW	3B	–	Text	See source	2023-11-29	chat assistant; compact
StableLM-Alpha	OW	7B	–	Text	See source	2023-04-19	open-source; conversation
Falcon — Technology Innovation Institute (TII) (Falcon 180B (Sep 2023)) [family page]							
Falcon-7B	OW	7B	–	Text	See source	2023-06-05	RefinedWeb dataset; high-quality pretraining
RedPajama — Together AI (RedPajama) [family page]							
RedPajama-INCITE-7B	OW	7B	2K	Text	See source	2023-05-05	Fully open training data; Reproducible
SOLAR — Upstage (SOLAR) [family page]							
SOLAR-10.7B	OW	10.7B	4K	Text	See source	2023-12-13	Depth Up-Scaling (DUS); Merged and scaled from Llama 2 architecture
GLM — Zhipu AI (GLM-5 (Feb 2026)) [family page]							
ChatGLM3-6B	OW	6B	–	Text	See source	2023-10-27	tool use; code interpreter
ChatGLM2-6B	OW	6B	–	Text	See source	2023-06-25	legacy
ChatGLM-6B	OW	6B	–	Text	See source	2023-03-14	bilingual; quantized for consumer GPUs

S18.5 Monitoring Sources and Maintenance

Table S13 lists the sources monitored to keep the inventory current between versions.

Table S13. High-value monitoring sources retained from the inventory files.

Source	Use
Hugging Face model hub and leaderboards	New releases, model cards, and benchmark submissions
Artificial Analysis	Context windows, pricing, and comparative capability snapshots
LLM Stats	Consolidated model metadata and API comparisons
LMSYS / Chatbot Arena	Human preference rankings
BentoML and vendor engineering blogs	Practical deployment write-ups and model updates
Official vendor changelogs	Release confirmation for OpenAI, Anthropic, Google, Meta, Mistral, Alibaba, and DeepSeek

Recommended maintenance queries from the inventory drafts included combinations such as “best small language model [year]”, “new open-weight reasoning model”, and vendor-specific searches around context window, model card, and release notes. In practice, the highest-signal workflow is to combine vendor documentation with model cards and a benchmark tracker rather than relying on any single static list.

References

[1] Marah Abdin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, Lingjiao Chen, Gustavo de Rosa, Suriya Gunasekar, Mojan Javaheripi, Neel Joshi, Piero Kauffmann, Yash Lara, Caio César Teodoro Mendes, Arindam Mitra, Besmira Nushi, Dimitris Papailiopoulos, Olli Saarikivi, Shital Shah, Vaishnavi Shrivastava, Vibhav Vineet, Yue Wu, Safoora Yousefi, and Guoqing Zheng. 2025. Phi-4-Reasoning Technical Report. arXiv:2504.21318. arXiv:2504.21318 https://arxiv.org/abs/2504.21318

[2] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Qin Cai, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Weizhu Chen, Yen-Chun Chen, Yi-Ling Chen, Hao Cheng, Parul Chopra, Xiyang Dai, Matthew Dixon, Ronen Eldan, Victor Fragoso, Jianfeng Gao, Mei Gao, et al. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219. arXiv:2404.14219 https://arxiv.org/abs/2404.14219

[3] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. 2024. Phi-4 Technical Report. arXiv:2412.08905. arXiv:2412.08905 https://arxiv.org/abs/2412.08905

[4] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. 2023. AutoMix: Automatically Mixing Language Models. arXiv:2310.12963. arXiv:2310.12963 https://arxiv.org/abs/2310.12963

[5] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245. arXiv:2305.13245 https://arxiv.org/abs/2305.13245

[6] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. arXiv:2502.02737. arXiv:2502.02737 https://arxiv.org/abs/2502.02737

[7] Apple. 2025. Apple Intelligence Foundation Language Models: Tech Report 2025. arXiv:2507.13575. arXiv:2507.13575 https://arxiv.org/abs/2507.13575

[8] Apple Inc. 2024. *Apple Intelligence Foundation Language Models.* Technical Report. Apple Machine Learning Research. https://arxiv.org/abs/2407.21075

[9] Apple Machine Learning Research. 2023. MLX: An Array Framework for Apple Silicon. Software, GitHub repository. https://github.com/ml-explore/mlx

[10] Apple Machine Learning Research. 2026. Introducing the Third Generation of Apple's Foundation Models. https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models.

[11] Viraat Aryabumi, John Dang, Dwarak Talupuru, Saurabh Dash, David Cairuz, Hangyu Lin, Bharat Venkitesh, Madeline Smith, Jon Ander Campos, Yi Chern Tan, Kelly Marchisio, Max Bartolo, Sebastian Ruder, Acyr Locatelli, Julia Kreutzer, Nick Frosst, Aidan Gomez, Phil Blunsom, Marzieh Fadaee, Ahmet Üstün, and Sara Hooker. 2024. Aya 23: Open Weight Releases to Further Multilingual Progress. arXiv:2405.15032. arXiv:2405.15032 https://arxiv.org/abs/2405.15032

[12] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732. arXiv:2108.07732 https://arxiv.org/abs/2108.07732

[13] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. arXiv:2502.13923. arXiv:2502.13923 https://arxiv.org/abs/2502.13923

[14] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlicek, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM3: smol, Multilingual, Long-Context Reasoner. Hugging Face blog, https://huggingface.co/blog/smollm3.

[15] Mohamad Ballout, Ulf Krumnack, Gunther Heidemann, and Kai-Uwe Kühnberger. 2024. Efficient Knowledge Distillation: Empowering Small Language Models with Teacher Model Insights. In *Natural Language Processing and Information Systems (NLDB 2024).* arXiv:2409.12586 doi:10.1007/978-3-031-70239-6_3

[16] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982. arXiv:2506.07982 https://arxiv.org/abs/2506.07982

[17] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. arXiv:2506.02153. Version 3, 22 September 2026. arXiv:2506.02153 https://arxiv.org/abs/2506.02153

[18] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, Thomas Unterthiner, Daniel Keysers, Skanda Koppula, Fangyu Liu, Adam Grycner, Alexey Gritsenko, Neil Houlsby, Manoj Kumar, Keran Rong, Julian Eisenschlos, Rishabh Kabra, Matthias Bauer, Matko Bošnjak, Xi Chen, Matthias Minderer, Paul Voigtlaender, Ioana Bica, Ivana Balazevic, Joan Puigcerver, Pinelopi Papalampidi, et al. 2024. PaliGemma: A Versatile 3B VLM for Transfer. arXiv:2407.07726. arXiv:2407.07726 https://arxiv.org/abs/2407.07726

[19] BigScience Workshop. 2022. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. arXiv:2211.05100. https://arxiv.org/abs/2211.05100

[20] Su Lin Blodgett, Solon Barocas, Hal Daumé III, and Hanna Wallach. 2020. Language (Technology) is Power: A Critical Survey of "Bias" in NLP. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics.* Association for Computational Linguistics, Online, 5454–5476. doi:10.18653/v1/2020.acl-main.485

[21] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. arXiv:2401.10774. arXiv:2401.10774 https://arxiv.org/abs/2401.10774

[22] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Martin Katz, and Nikolaos Aletras. 2022. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL).* arXiv:2110.00976 https://arxiv.org/abs/2110.00976

[23] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318. arXiv:2302.01318 https://arxiv.org/abs/2302.01318

[24] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176. arXiv:2305.05176 https://arxiv.org/abs/2305.05176

[25] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert,

Fotios Chantzis, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374. arXiv:2107.03374 https://arxiv.org/abs/2107.03374

[26] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457. arXiv:1803.05457 https://arxiv.org/abs/1803.05457

[27] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168. arXiv:2110.14168 https://arxiv.org/abs/2110.14168

[28] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. doi:10.18653/v1/2024.acl-long.70

[29] Alex de Vries. 2023. The Growing Energy Footprint of Artificial Intelligence. *Joule* 7, 10 (2023), 2191–2194. doi:10.1016/j.joule.2023.09.004

[30] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. https://arxiv.org/abs/2501.12948

[31] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339. arXiv:2208.07339 https://arxiv.org/abs/2208.07339

[32] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., New Orleans, Louisiana, USA. arXiv:2305.14314 doi:10.52202/075280-0441

[33] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. arXiv:2404.14618. arXiv:2404.14618 https://arxiv.org/abs/2404.14618

[34] Ronen Eldan and Yuanzhi Li. 2023. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759. https://arxiv.org/abs/2305.07759

[35] Cooper Elsworth, Keguo Huang, David Patterson, Ian Schneider, Robert Sedivy, Savannah Goodman, Ben Townsend, Parthasarathy Ranganathan, Jeff Dean, Amin Vahdat, Ben Gomes, and James Manyika. 2025. Measuring the environmental impact of delivering AI at Google Scale. arXiv:2508.15734. arXiv:2508.15734 https://arxiv.org/abs/2508.15734

[36] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23 (2022), 1–39. arXiv:2101.03961 https://arxiv.org/abs/2101.03961

[37] Tao Feng, Yicheng Li, Li Chenglin, Hao Chen, Fei Yu, and Yin Zhang. 2024. Teaching Small Language Models Reasoning through Counterfactual Distillation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 5831–5842. doi:10.18653/v1/2024.emnlp-main.333

[38] Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, Hawaii, USA, 10323–10337. https://proceedings.mlr.press/v202/frantar23a.html

[39] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *11th International Conference on Learning Representations (ICLR)*. OpenReview.net, Kigali, Rwanda. https://openreview.net/forum?id=tcbBPnfwxS

[40] Yao Fu, Yin Yu, Xiaotian Han, Runchao Li, Xianxuan Long, Haotian Yu, and Pan Li. 2024. Dynamic Self-Distillation via Previous Mini-batches for Fine-tuning Small Language Models. arXiv:2411.16991. arXiv:2411.16991 https://arxiv.org/abs/2411.16991

[41] Leo Gao, Jonathan Tow, Baber Abbasi, et al. 2023. A Framework for Few-Shot Language Model Evaluation. Zenodo. https://github.com/EleutherAI/lm-evaluation-harness

[42] Gemini Team, Google. 2023. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805. arXiv:2312.11805 https://arxiv.org/abs/2312.11805

[43] Gemma Team, Google DeepMind. 2024. Gemma 2: Improving Open Language Models at a Practical Size. arXiv:2408.00118. arXiv:2408.00118 https://arxiv.org/abs/2408.00118

[44] Gemma Team, Google DeepMind. 2024. Gemma: Open Models Based on Gemini Research and Technology. arXiv:2403.08295. arXiv:2403.08295 https://arxiv.org/abs/2403.08295

[45] Gemma Team, Google DeepMind. 2025. Gemma 3 Technical Report. arXiv:2503.19786. https://arxiv.org/abs/2503.19786

[46] Gemma Team, Google DeepMind. 2026. Gemma 4 Technical Report. arXiv:2607.02770. Google DeepMind; models released 2 April 2026 (E2B–31B, dense and MoE). arXiv:2607.02770 https://arxiv.org/abs/2607.02770

[47] Georgi Gerganov. 2023. llama.cpp. GitHub repository: https://github.com/ggerganov/llama.cpp.

[48] Google. 2025. Gemma 3n: On-Device Multimodal Models. Google AI Blog. https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/

[49] Google and Apple. 2026. Joint Statement from Google and Apple. https://blog.google/company-news/inside-google/company-announcements/joint-statement-google-apple/. Published 12 January 2026.

[50] Google Developers. 2025. LiteRT with QNN Acceleration. Google AI Blog. https://ai.google.dev/edge/litert

[51] Albert Gu and Tri Dao. 2023. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752. arXiv:2312.00752 https://arxiv.org/abs/2312.00752

[52] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, Aditya Narayana, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel N. Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory M. Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John Nay, Jonathan H. Choi, Kevin Tobia, Margaret Hagan, Megan Ma, Michael Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny Gandhi, Tom Zur, Varun Iyer, and Zehua Li. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2308.11462 https://arxiv.org/abs/2308.11462

[53] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. 2023. Textbooks Are All You Need. arXiv:2306.11644. arXiv:2306.11644 https://arxiv.org/abs/2306.11644

[54] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196. https://arxiv.org/abs/2401.14196

[55] Mohammad Junayed Hasan, Fuad Rahman, and Nabeel Mohammed. 2025. OptimCLM: Optimizing Clinical Language Models for Predicting Patient Outcomes via Knowledge Distillation, Pruning, and Quantization. *International Journal of Medical Informatics* 195 (2025), 105764. doi:10.1016/j.ijmedinf.2024.105764

[56] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *9th International Conference on Learning Representations (ICLR)*. OpenReview.net. https://openreview.net/forum?id=d7KBjmI3GmQ

[57] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2103.06268 https://arxiv.org/abs/2103.06268

[58] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, Vol. 34. Curran Associates, Inc. arXiv:2103.03874 https://arxiv.org/abs/2103.03874

[59] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531. arXiv:1503.02531 https://arxiv.org/abs/1503.02531

[60] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *10th International Conference on Learning Representations (ICLR)*. OpenReview.net. https://openreview.net/forum?id=nZeVKeeFYf9

[61] IBM Granite Team. 2025. IBM Granite 4.0: Hyper-Efficient, High-Performance Hybrid Models for Enterprise. https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models.

[62] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825. arXiv:2310.06825 https://arxiv.org/abs/2310.06825

[63] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. arXiv:2401.04088. https://arxiv.org/abs/2401.04088

[64] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for Natural Language Understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 4163–4174. doi:10.18653/v1/2020.findings-emnlp.372

[65] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for*

Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Vancouver, Canada, 1601–1611. doi:10.18653/v1/P17-1147

[66] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. arXiv:2001.08361. arXiv:2001.08361 https://arxiv.org/abs/2001.08361

[67] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. 2025. Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation. arXiv:2510.11977. arXiv:2510.11977 https://arxiv.org/abs/2510.11977

[68] Hyunjae Kim, Hyeon Hwang, Jiwoo Lee, Sihyeon Park, Dain Kim, Taewhoo Lee, Chanwoong Yoon, Jiwoong Sohn, Jungwoo Park, Olga Reykhart, Thomas Fetherston, Donghee Choi, Soo Heon Kwak, Qingyu Chen, and Jaewoo Kang. 2025. Small Language Models Learn Enhanced Reasoning Skills from Medical Textbooks. *npj Digital Medicine* 8 (2025), 240. doi:10.1038/s41746-025-01653-8

[69] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. 2024. SqueezeLLM: Dense-and-Sparse Quantization. arXiv:2306.07629. arXiv:2306.07629 https://arxiv.org/abs/2306.07629

[70] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, Koblenz, Germany, 611–626. doi:10.1145/3600006.3613165

[71] Md Tahmid Rahman Laskar, Xue-Yong Fu, and Shashi Bhushan TN. 2026. SFT or RL for Tool-Calling Agents? A Controlled Study Across Data, Method, and Scale. arXiv:2609.17848. arXiv:2609.17848 https://arxiv.org/abs/2609.17848

[72] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, Hawaii, USA, 19274–19286. https://proceedings.mlr.press/v202/leviathan23a.html

[73] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[74] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks Are All You Need II: phi-1.5 technical report. arXiv:2309.05463. https://arxiv.org/abs/2309.05463

[75] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. arXiv:2401.15077. arXiv:2401.15077 https://arxiv.org/abs/2401.15077

[76] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* (2023). https://openreview.net/forum?id=iO4LZibEqW

[77] Huanxuan Liao, Shizhu He, Yao Xu, Yuanzhe Zhang, Kang Liu, and Jun Zhao. 2025. Neural-Symbolic Collaborative Distillation: Advancing Small Language Models for Complex Reasoning Tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*. arXiv:2409.13203 doi:10.1609/aaai.v39i23.34636

[78] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. 2024. Jamba: A Hybrid Transformer-Mamba Language Model. arXiv:2403.19887. arXiv:2403.19887 https://arxiv.org/abs/2403.19887

[79] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, Vol. 6. MLSys, Santa Clara, California, USA. https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[80] Alexander H. Liu, Andy Ehrenberg, Andy Lo, Clément Denoix, Corentin Barreau, Guillaume Lample, Jean-Malo Delignon, Khyathi Raghavi Chandu, Patrick von Platen, Pavankumar Reddy Muddireddy, Sanchit Gandhi, Soham Ghosh, Srijan Mishra, Thomas Foubert, Abhinav Rastogi, Adam Yang, Albert Q. Jiang, Alexandre Sablayrolles, Amélie Héliou, Amélie Martin, Anmol Agarwal, Antoine Roux, Arthur Darcet, Arthur Mensch, Baptiste Bout, Baptiste Rozière, Baudouin De Monicault, Chris Bamford, Christian Wallenwein, Christophe Renaudin, Clémence Lanfranchi, Darius

Dabert, Devendra Singh Chaplot, Devon Mizelle, Diego de las Casas, Elliot Chane-Sane, Emilien Fugier, Emma Bou Hanna, Gabrielle Berrada, Gauthier Delerce, Gauthier Guinet, Georgii Novikov, Guillaume Martin, Himanshu Jaju, Jan Ludziejewski, Jason Rute, Jean-Hadrien Chabran, Jessica Chudnovsky, Joachim Studnia, Joep Barmentlo, Jonas Amar, Josselin Somerville Roberts, Julien Denize, Karan Saxena, Karmesh Yadav, Kartik Khandelwal, Kush Jain, Lélio Renard Lavaud, Léonard Blier, Lingxiao Zhao, Louis Martin, Lucile Saulnier, Luyu Gao, Marie Pellat, Mathilde Guillaumin, Mathis Felardos, Matthieu Dinot, Maxime Darrin, Maximilian Augustin, Mickaël Seznec, Neha Gupta, Nikhil Raghuraman, Olivier Duchenne, Patricia Wang, Patryk Saffer, Paul Jacob, Paul Wambergue, Paula Kurylowicz, Philomène Chagniot, Pierre Stock, Pravesh Agrawal, Rémi Delacourt, Romain Sauvestre, Roman Soletskyi, Sagar Vaze, Sandeep Subramanian, Saurabh Garg, Shashwat Dalal, Siddharth Gandhi, Sumukh Aithal, Szymon Antoniak, Teven Le Scao, Thibault Schueller, Thibaut Lavril, Thomas Robert, Thomas Wang, Timothée Lacroix, Tom Bewley, Valeriia Nemychnikova, Victor Paltz, Virgile Richard, Wen-Ding Li, William Marshall, Xuanyu Zhang, Yihan Wan, and Yunhao Tang. 2025. Voxtral. arXiv:2507.13264. arXiv:2507.13264 https://arxiv.org/abs/2507.13264

[81] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Vienna, Austria. arXiv:2308.03688 https://arxiv.org/abs/2308.03688

[82] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. arXiv:2407.21783. https://arxiv.org/abs/2407.21783

[83] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173. arXiv:2402.19173 https://arxiv.org/abs/2402.19173

[84] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Wei Liu, Jian Luan, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. 2025. Demystifying Small Language Models for Edge Deployment. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 14747–14764. doi:10.18653/v1/2025.acl-long.718

[85] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. 2024. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764. arXiv:2402.17764 https://arxiv.org/abs/2402.17764

[86] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. LLM-Pruner: On the Structural Pruning of Large Language Models. arXiv:2305.11627. arXiv:2305.11627 https://arxiv.org/abs/2305.11627

[87] Maginative. 2023. Samsung Unveils Gauss, Its New Generative AI System. News report on the Samsung AI Forum 2023 announcement. Accessed 3 October 2026. Samsung has published no technical report, parameter count, or architecture for Gauss. https://www.maginative.com/article/samsung-unveils-gauss-its-new-generative-ai-system/

[88] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. ShortGPT: Layers in Large Language Models are More Redundant Than You Expect. arXiv:2403.03853. arXiv:2403.03853 https://arxiv.org/abs/2403.03853

[89] Meta AI. 2024. ExecuTorch: End-to-End Solution for Enabling On-Device AI. PyTorch Blog. https://pytorch.org/executorch/

[90] Meta AI. 2024. Llama 3.2 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md. Retrieved 8 October 2026.

[91] Meta AI. 2024. Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/. Published 25 September 2024.

[92] Meta AI. 2025. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI Innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

[93] Microsoft. 2019. ONNX Runtime. GitHub repository: https://github.com/microsoft/onnxruntime.

[94] Microsoft. 2024. Phi-3.5-mini-instruct Model Card. https://huggingface.co/microsoft/Phi-3.5-mini-instruct. Phi-3.5 is also described in the later versions of the Phi-3 technical report.

[95] Microsoft. 2025. Phi-4-mini-instruct Model Card. https://huggingface.co/microsoft/Phi-4-mini-instruct. Retrieved 8 October 2026.

[96] Microsoft, Abdelrahman Abouelenin, Atabak Ashfaq, Adam Atkinson, Hany Awadalla, Nguyen Bach, Jianmin Bao, Alon Benhaim, Martin Cai, Vishrav Chaudhary, Congcong Chen, Dong Chen, Dongdong Chen, Junkun Chen, Weizhu Chen, Yen-Chun Chen, Yi ling Chen, Qi Dai, Xiyang Dai, Ruchao Fan, Mei Gao, Min Gao, Amit Garg, Abhishek Goswami, Junheng Hao, Amr Hendy, Yuxuan Hu, Xin Jin, Mahmoud Khademi, Dongwoo Kim, et al. 2025. Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs. arXiv:2503.01743. arXiv:2503.01743 https://arxiv.org/abs/2503.01743

[97] Microsoft Research. 2023. Phi-2: The Surprising Power of Small Language Models. Microsoft Research Blog. https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/
[98] Mistral AI. 2024. Codestral: Hello, World! Mistral AI Blog. https://mistral.ai/news/codestral
[99] Arindam Mitra, Luciano Del Corro, Shweti Mahajan, Andres Codas, Clarisse Simoes, Sahaj Agarwal, Xuxi Chen, Anastasia Razdaibiedina, Erik Jones, Kriti Aggarwal, Hamid Palangi, Guoqing Zheng, Corby Rosset, Hamed Khanpour, and Ahmed Awadallah. 2023. Orca 2: Teaching Small Language Models How to Reason. arXiv:2311.11045. https://arxiv.org/abs/2311.11045
[100] MLC Team. 2024. MLC LLM: Universal LLM Deployment Engine. GitHub repository: https://github.com/mlc-ai/mlc-llm.
[101] MLC Team. 2024. WebLLM: High-Performance In-Browser LLM Inference Engine. GitHub repository. https://arxiv.org/abs/2412.15803
[102] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. 2023. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv:2306.02707. arXiv:2306.02707 https://arxiv.org/abs/2306.02707
[103] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. 2024. Compact Language Models via Pruning and Knowledge Distillation. Advances in Neural Information Processing Systems (NeurIPS); arXiv:2407.14679. doi:10.52202/079017-1299
[104] Tung Nguyen and Tuyen Nguyen. 2025. An Evaluation of LLMs Inference on Popular Single-board Computers. arXiv:2511.07425. arXiv:2511.07425 https://arxiv.org/abs/2511.07425
[105] NVIDIA. 2025. NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model. arXiv:2508.14444. arXiv:2508.14444 https://arxiv.org/abs/2508.14444
[106] NVIDIA Corporation. 2024. TensorRT-LLM: Accelerating Large Language Model Inference. NVIDIA Developer documentation. https://developer.nvidia.com/tensorrt
[107] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations (ICLR)*. arXiv:2406.18665 https://arxiv.org/abs/2406.18665
[108] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR. https://proceedings.mlr.press/v267/patil25a.html
[109] David Patterson, Joseph Gonzalez, Urs Hölzle, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2022. The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. *Computer* 55, 7 (2022), 18–28. doi:10.1109/MC.2022.3148714
[110] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. Carbon Emissions and Large Neural Network Training. arXiv:2104.10350. arXiv:2104.10350 https://arxiv.org/abs/2104.10350
[111] Qualcomm Technologies. 2024. On-Device AI: Enabling the Next Generation of Mobile Intelligence. Qualcomm white paper. https://www.qualcomm.com/research/artificial-intelligence/on-device-ai
[112] Qwen Team, Alibaba Group. 2024. Qwen2.5 Technical Report. arXiv:2412.15115. arXiv:2412.15115 https://arxiv.org/abs/2412.15115
[113] Qwen Team, Alibaba Group. 2026. Qwen3.5 Small Models for On-Device Applications. https://qwen.ai/blog?id=qwen3.5. Alibaba; 0.8B–9B dense multimodal, Gated DeltaNet hybrid, released 2 March 2026.
[114] Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know What You Don't Know: Unanswerable Questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, Melbourne, Australia, 784–789. doi:10.18653/v1/P18-2124
[115] Liliang Ren, Congcong Chen, Haoran Xu, Young Jin Kim, Adam Atkinson, Zheng Zhan, Jiankai Sun, Baolin Peng, Liyuan Liu, Shuohang Wang, Hao Cheng, Jianfeng Gao, Weizhu Chen, and Yelong Shen. 2025. Decoder-Hybrid-Decoder Architecture for Efficient Reasoning with Long Generation. In *Advances in Neural Information Processing Systems*. Introduces the SambaY architecture (Phi-4-mini-flash). arXiv:2507.06607 doi:10.52202/085713-3591
[116] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. 2015. FitNets: Hints for Thin Deep Nets. arXiv:1412.6550. arXiv:1412.6550 https://arxiv.org/abs/1412.6550
[117] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. arXiv:2308.12950. arXiv:2308.12950 https://arxiv.org/abs/2308.12950

[118] Chenyang Shao, Xinyuan Hu, Yutang Lin, and Fengli Xu. 2025. Division-of-Thoughts: Harnessing Hybrid Language Model Synergy for Efficient On-Device Agents. In *Proceedings of the ACM Web Conference 2025*. arXiv:2502.04392 doi:10.1145/3696410.3714765

[119] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *5th International Conference on Learning Representations (ICLR)*. OpenReview.net, Toulon, France. https://openreview.net/forum?id=B1ckMDqlg

[120] Philipp Singer, Pascal Pfeiffer, Yauhen Babakhin, Maximilian Jeblick, Nischay Dhankhar, Gabor Fodor, and Sri Satish Ambati. 2024. H2O-Danube-1.8B Technical Report. arXiv:2401.16818. arXiv:2401.16818 https://arxiv.org/abs/2401.16818

[121] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Seattle, Washington, USA, 1631–1642. doi:10.18653/v1/D13-1170

[122] Sharath Turuvekere Sreenivas, Saurav Muralidharan, Raviraj Joshi, Marcin Chochowski, Ameya Sunil Mahabaleshwarkar, Gerald Shen, Jiaqi Zeng, Zijia Chen, Yoshi Suhara, Shizhe Diao, Chenhan Yu, Wei-Chun Chen, Hayley Ross, Oluwatobi Olabiyi, Ashwath Aithal, Oleksii Kuchaiev, Daniel Korzekwa, Pavlo Molchanov, Mostofa Patwary, Mohammad Shoeybi, Jan Kautz, and Bryan Catanzaro. 2024. LLM Pruning and Distillation in Practice: The Minitron Approach. arXiv:2408.11796. arXiv:2408.11796 https://arxiv.org/abs/2408.11796

[123] Gaurav Srivastava, Shuxiang Cao, and Xuan Wang. 2025. Towards Reasoning Ability of Small Language Models. arXiv:2502.11569. arXiv:2502.11569 https://arxiv.org/abs/2502.11569

[124] Stability AI. 2024. Stable LM 2 1.6B Technical Report. arXiv:2402.17834. arXiv:2402.17834 https://arxiv.org/abs/2402.17834

[125] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and Policy Considerations for Deep Learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 3645–3650. doi:10.18653/v1/P19-1355

[126] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2024. A Simple and Effective Pruning Approach for Large Language Models. In *12th International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria. https://openreview.net/forum?id=PxoFut3dWW

[127] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Allyson Ettinger, Michal Guerquin, David Heineman, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, et al. 2025. 2 OLMo 2 Furious. arXiv:2501.00656. arXiv:2501.00656 https://arxiv.org/abs/2501.00656

[128] Technology Innovation Institute. 2025. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance. arXiv:2507.22448. https://arxiv.org/abs/2507.22448

[129] Jiacheng Wang, Yejun Zeng, Jinyang Guo, Yuqing Ma, Aishan Liu, and Xianglong Liu. 2025. SLMQuant: Benchmarking Small Language Model Quantization for Practical Deployment. arXiv:2511.13023. arXiv:2511.13023 doi:10.1145/3746262.3761973

[130] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, Inc., New Orleans, Louisiana, USA. arXiv:2201.11903 doi:10.52202/068431-1800

[131] Adina Williams, Nikita Nangia, and Samuel R. Bowman. 2018. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, New Orleans, Louisiana, 1112–1122. doi:10.18653/v1/N18-1101

[132] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2023. Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning. arXiv:2310.06694. arXiv:2310.06694 https://arxiv.org/abs/2310.06694

[133] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2022. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438. arXiv:2211.10438 https://arxiv.org/abs/2211.10438

[134] Haoran Xu, Baolin Peng, Hany Awadalla, Dongdong Chen, Yen-Chun Chen, Mei Gao, Young Jin Kim, Yunsheng Li, Liliang Ren, Yelong Shen, Shuohang Wang, Weijian Xu, Jianfeng Gao, and Weizhu Chen. 2025. Phi-4-Mini-Reasoning: Exploring the Limits of Small Reasoning Language Models in Math. arXiv:2504.21233. arXiv:2504.21233 https://arxiv.org/abs/2504.21233

[135] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. 2024. On-Device Language Models: A Comprehensive Review. arXiv:2409.00088. arXiv:2409.00088 https://arxiv.org/abs/2409.00088

[136] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. 2024. A Survey on Knowledge Distillation of Large Language Models. arXiv:2402.13116. arXiv:2402.13116 https://arxiv.org/abs/2402.13116

[137] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388. arXiv:2505.09388 https://arxiv.org/abs/2505.09388

[138] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, et al. 2024. Qwen2 Technical Report. arXiv:2407.10671. https://arxiv.org/abs/2407.10671

[139] Chuanpeng Yang, Wang Lu, Yao Zhu, Yidong Wang, Qian Chen, Chenlong Gao, Bingjie Yan, and Yiqiang Chen. 2025. Survey on Knowledge Distillation for Large Language Models: Methods, Evaluation, and Application. *ACM Transactions on Intelligent Systems and Technology* (2025). doi:10.1145/3699518

[140] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. In *International Conference on Learning Representations (ICLR)*. arXiv:2412.06464 https://arxiv.org/abs/2412.06464

[141] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045. arXiv:2406.12045 https://arxiv.org/abs/2406.12045

[142] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 4791–4800. doi:10.18653/v1/P19-1472

[143] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385. https://arxiv.org/abs/2401.02385

[144] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Chen Xu, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. 2025. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *Computational Linguistics* (2025). doi:10.1162/coli.a.16

[145] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, Vol. 36. Curran Associates, Inc., New Orleans, Louisiana, USA. arXiv:2306.05685 doi:10.52202/075280-2020

[146] Zihan Zhou, Simon Kurz, and Zhixue Zhao. 2025. Revisiting Pruning vs Quantization for Small Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, Suzhou, China, 12055–12070. doi:10.18653/v1/2025.findings-emnlp.645

[147] Ahmet Üstün, Viraat Aryabumi, Zheng-Xin Yong, Wei-Yin Ko, Daniel D'souza, Gbemileke Onilude, Neel Bhandari, Shivalika Singh, Hui-Lee Ooi, Amr Kayid, Freddie Vargus, Phil Blunsom, Shayne Longpre, Niklas Muennighoff, Marzieh Fadaee, Julia Kreutzer, and Sara Hooker. 2024. Aya Model: An Instruction Finetuned Open-Access Multilingual Language Model. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. doi:10.18653/v1/2024.acl-long.845